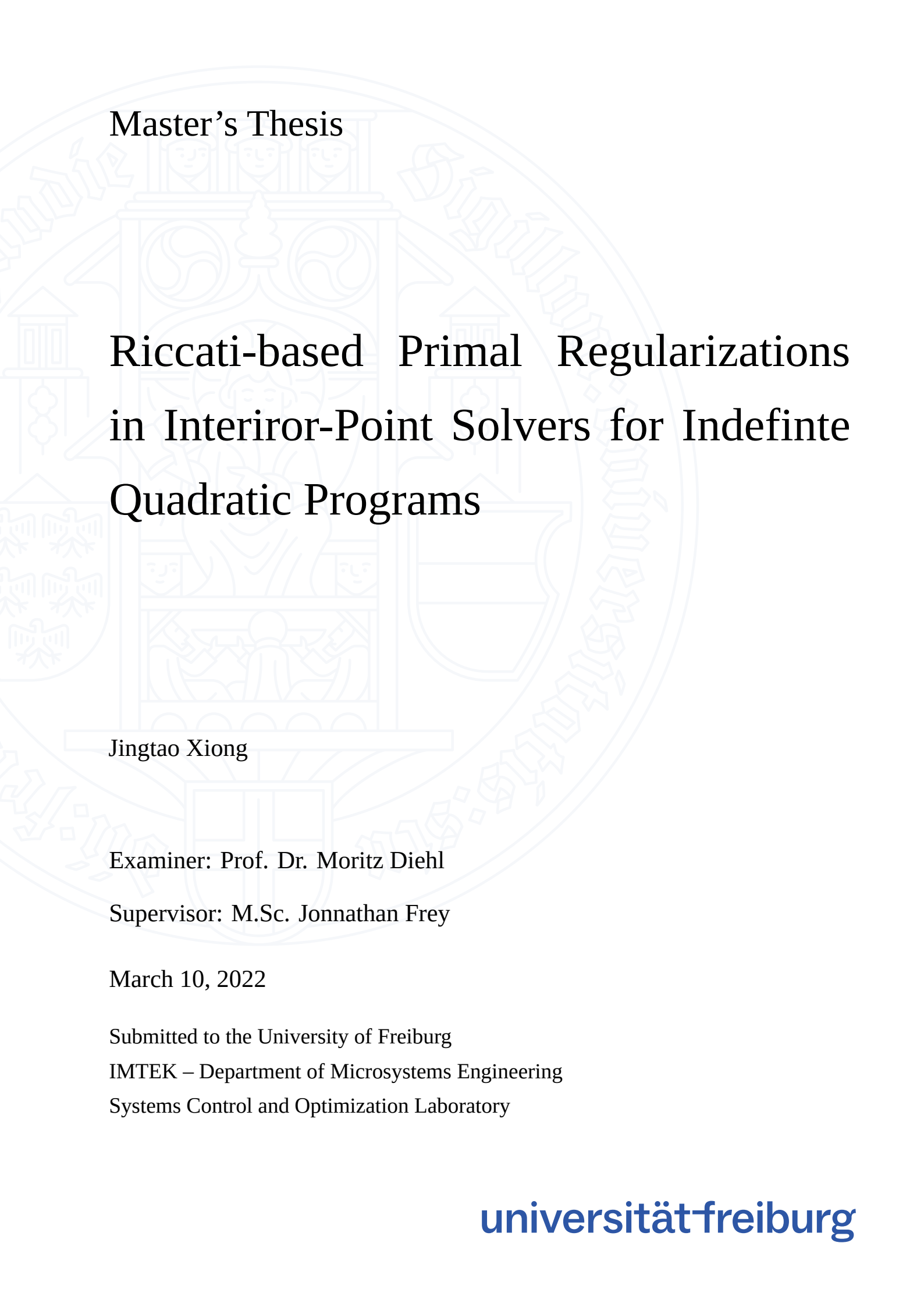




Master's Thesis

Riccati-based Primal Regularizations in Interior-Point Solvers for Indefinite Quadratic Programs

Jingtao Xiong

Examiner: Prof. Dr. Moritz Diehl

Supervisor: M.Sc. Jonnathan Frey

March 10, 2022

Submitted to the University of Freiburg

IMTEK – Department of Microsystems Engineering

Systems Control and Optimization Laboratory

University of Freiburg
IMTEK – Department of Microsystems Engineering
Systems Control and Optimization Laboratory

Author	Jingtao Xiong, Matriculation Number: 5986123
Editing Time	July 13, 2021 - March 10, 2022
Examiners	Prof. Dr. Moritz Diehl, IMTEK – Department of Microsystems Engineering Systems Control and Optimization Laboratory Prof. Dr. Max Doe, IMTEK – Department of Microsystems Engineering Laboratory for something that likely has a long title as well
Supervisor	M.Sc. Jonnathan Frey, IMTEK – Department of Microsystems Engineering Systems Control and Optimization Laboratory
Declaration	I hereby declare, that I am the sole author and composer of this Thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work. I hereby also declare, that my Thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Place, Date

Signature

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Contributions	2
1.3	Related Solvers	2
1.4	Outline	3
2	Primal–Dual Interior Point Solver in Optimal Control Problems	5
2.1	Primal–Dual Interior Point Method	5
2.2	Efficient Solution with Riccati Recursion	7
2.3	Adaptive Barrier Update	9
2.3.1	Mehrotra’s Predictor-Corrector	9
2.3.2	Mehrotra’s Probing	11
2.4	Step Size Control	11
2.4.1	Fraction-to-the-Boundary Rule	11
2.4.2	Two-Phase Line Search with Backtracking	12
3	Primal Regularization for Indefinite Quadratic Programs	15
3.1	Primal regularization with Riccati-based Detection	15
3.1.1	Regularization Strategies	16
3.1.2	Full Hessian regularization	18
3.1.3	Reduced Hessian regularization	18
3.1.4	Block Reduced Hessian From Riccati Recursion	19
3.2	Riccati-based Primal Regularization	20
3.2.1	Stage Regularization in Riccati Recursion	21
3.2.2	Failure of Minimum-Perturbation Regularization	22
3.2.3	Insights and Proposed Methods	23
3.3	Asymptotic Inactivity of the Regularization	24

4	A Benchmark for Indefinite OCP Quadratic Programs	29
4.1	Benchmark Construction	29
4.1.1	Formulation of QP subproblems from OCPs	30
4.1.2	Benchmark OCPs	32
4.1.3	Collecting Strategy	34
4.2	Benchmark Analysis	35
4.2.1	Distribution of Indefiniteness	35
4.2.2	Source of Indefiniteness	38
5	Numerical Experiments	41
5.1	Experiment Metrics	41
5.1.1	Iteration Profile	42
5.1.2	Flop Unit Estimation	42
5.2	Prototype Solver Implementation	43
5.3	Experiment Setup	45
5.3.1	Experimented Regularizations	45
5.3.2	General setup	48
5.4	Numerical Comparison of Primal Regularizations	49
5.4.1	Comparison among Riccati-based regularizations	49
5.4.2	Comparison with Full Hessian Regularizations	52
5.4.3	Comparison with Inertia Correction	53
5.5	Discussion	55
5.5.1	Failure by status	56
5.5.2	Failure by category	57
6	Conclusion and Outlook	61
	Bibliography	63

Chapter 1

Introduction

1.1 Motivation

In the field of optimal control, direct multiple shooting [1] is a widely used method for solving continuous-time optimal control problems, which transcribes the OCP into a nonlinear programming problem by discretizing the time horizon into multiple shooting intervals [2].

This NLP is commonly solved by SQP, which computes each step from a QP subproblem which inherits an optimal control structure that Riccati recursion can exploit [3]. To retain the fast local convergence of SQP, the QP subproblems are often formulated with the exact Hessian of the Lagrangian [4], this could result in an indefinite Hessian thus a nonconvex QP as subproblem, which is not guaranteed to have a unique solution and can cause the Riccati recursion to fail.

A common remedy is to regularize at the SQP level, i.e. to modify the QP Hessian before the QP is passed to the solver. Such a correction is decided from the Hessian of the QP subproblem alone and therefore overlooks two aspects. First, inside an interior point method, the barrier term of the inequality constraints adds curvature to the Hessian, which can render the system well-defined even when the QP Hessian itself is indefinite. Second, when the correction targets the full Hessian, it ignores that positive definiteness of the reduced Hessian is already sufficient for the QP to have a unique solution.

This thesis therefore regularizes at the QP level, inside the solver, where the correction is computed from the factorized matrices and applied only when indefiniteness is actually detected.

1.2 Contributions

The main contributions of this thesis are the following:

1. **Riccati-based primal regularization**, which regularizes the reduced Hessian blocks obtained from the Riccati recursion, only at the stages where the Cholesky factorization fails (Section 3.2).
2. **An improved stage regularization strategy**, derived from showing that minimum perturbation propagates negative curvature through the cost-to-go matrix to the preceding stages (Section ??).
3. **A proof of asymptotic inactivity**, so that the local convergence rate of the interior point method is preserved (Section 3.3).
4. **A benchmark of 175 indefinite OCP QPs** collected from the SQP iterations of eight OCPs, with an analysis of where the indefiniteness originates (Chapter 4).
5. **A numerical comparison** on this benchmark, in which the best Riccati-based method leaves 4 instances unsolved, against 32 to 59 for full Hessian regularization and 15 for inertia correction (Section 5.4).

1.3 Related Solvers

This thesis builds on two open-source software packages for embedded optimal control, `acados` and HPIPM.

`acados` [5] is a modular framework for fast NMPC. It transcribes the OCP by direct multiple shooting, solves the resulting NLP by SQP or its real-time iteration variant, and passes the QP subproblems to one of several interfaced QP solvers. When the exact Hessian is used, `acados` handles indefiniteness at the SQP level: the QP Hessian can be regularized before the QP is passed to the solver or be replaced by a Gauss–Newton approximation. In this thesis, `acados` is used to generate the SQP iterates from which the benchmark QPs are collected (Section 4.1).

HPIPM [6] is a primal–dual interior point solver for OCP-structured QPs and the default QP solver in `acados`. It solves the KKT system of each iteration by Riccati recursion, with a cost linear in the horizon length, and relies on high-performance linear algebra routines tailored to the small and medium-sized matrices of optimal control. HPIPM assumes a convex QP: the Riccati recursion requires the reduced Hessian blocks to be positive definite, so it fails on indefinite

QPs. This failure is precisely the criterion by which the benchmark QPs are collected (Section 4.1), and the prototype solver used in the experiments reproduces the iteration behavior of HPIPM on convex QPs, so that the regularization is the only difference between the two (Section 5.2).

1.4 Outline

The remainder of this thesis is organized as follows. Chapter 2 introduces the primal–dual interior point method for OCP-structured QPs, the Riccati recursion used to solve its KKT system, and the barrier update and step size control strategies adopted in the solver. Chapter 3 classifies primal regularization by its target matrix, develops the Riccati-based primal regularization, derives the improved stage regularization strategy from the failure of minimum perturbation, and proves that the regularization is asymptotically inactive. Chapter 4 constructs the benchmark of indefinite QPs from the SQP iterations of eight OCPs and analyzes how the indefiniteness is distributed over the stages and where it originates. Chapter 5 describes the prototype solver and the experiment setup, compares the Riccati-based methods with full Hessian regularization and inertia correction on the benchmark, and discusses the instances on which the methods fail. Finally, Chapter 6 summarizes the results and outlines directions for future work.

Chapter 2

Primal–Dual Interior Point Solver in Optimal Control Problems

In this chapter, the theoretical background regarding relevant techniques implemented in the solver are introduced. The primal-dual interior point method (IPM) is introduced first in Section 2.1 as the core algorithm for solving the optimal control problems in this thesis. Then, the Riccati recursion is introduced for efficiently solving the KKT system of the optimal control problem in Section 2.1. Finally, the adaptive barrier update and step size control strategies are discussed in Section 2.3 and Section 2.4 considering the nonconvex feature in this work.

2.1 Primal–Dual Interior Point Method

Consider a QP with equality $Fy = b$ and inequality constraints $Gy \leq d$. By introducing slack variables t , we have the following formulation:

$$\begin{aligned} & \underset{y, t}{\text{minimize}} && \frac{1}{2}y^\top Hy + g^\top y \\ & \text{subject to} && \begin{aligned} & Fy = b, \\ & Gy + t = d, \\ & t \geq 0, \end{aligned} \end{aligned} \tag{2.1}$$

we can further obtain the lagrangian function of the QP problem (2.1) as

$$\mathcal{L}(y, t, \pi, \lambda) = \frac{1}{2}y^\top Hy + g^\top y + \pi^\top (Fy - b) + \lambda^\top (Gy + t - d), \tag{2.2}$$

where π and $\lambda \geq 0$ are the Lagrange multipliers associated with the equality and inequality constraints, respectively. Then the KKT conditions of the QP

problem (2.1) are given by

$$Hy + g + F^\top \pi + G^\top \lambda = 0, \quad (2.3a)$$

$$Fy - b = 0, \quad (2.3b)$$

$$Gy + t - d = 0, \quad (2.3c)$$

$$\lambda_j t_j = 0, \quad j = 1, \dots, n_c, \quad (2.3d)$$

$$\lambda \geq 0, \quad t \geq 0. \quad (2.3e)$$

where n_c is the number of inequality constraints. The definiteness of the Hessian matrix H distinguishes the QP problem (2.1) as convex or nonconvex. In primal-dual IPM, a homotopy parameter τ , also called as barrier parameter, is applied to the only nonlinear equation (2.3d) for relaxing the complementarity, the equation (2.3d) can be rewritten as:

$$\lambda_j t_j = \tau, \quad j = 1, \dots, n_c, \quad (2.4)$$

the parameter τ is shrunk toward zero as the iterations approach the solution with certain strategy, and size step control is applied to ensure the satisfaction of inequality (2.3e). The corresponding update strategy and step size control will be discussed in the following sections.

In order to solve the nonlinear system of equations (2.3), the Newton's method is applied to the linearized system at each iteration i to obtain the step $(\Delta y, \Delta \pi, \Delta \lambda, \Delta t)$, which takes the form of:

$$\begin{bmatrix} H & F^\top & G^\top & 0 \\ F & 0 & 0 & 0 \\ G & 0 & 0 & I \\ 0 & 0 & T_i & \Lambda_i \end{bmatrix} \begin{bmatrix} \Delta y \\ \Delta \pi \\ \Delta \lambda \\ \Delta t \end{bmatrix} = - \begin{bmatrix} r_{\text{stat}}^i \\ r_{\text{eq}}^i \\ r_{\text{ineq}}^i \\ r_{\text{comp}}^i \end{bmatrix}, \quad (2.5)$$

With the diagonal matrices $T_i = \text{diag}(t_i)$, $\Lambda_i = \text{diag}(\lambda_i)$, and the perturbed KKT residuals follow as:

$$\begin{bmatrix} r_{\text{stat}}^i \\ r_{\text{eq}}^i \\ r_{\text{ineq}}^i \\ r_{\text{comp}}^i \end{bmatrix} := \begin{bmatrix} Hy_i + g + F^\top \pi_i + G^\top \lambda_i \\ Fy_i - b \\ Gy_i + t_i - d \\ T_i \Lambda_i \mathbf{1} - \tau_i \mathbf{1} \end{bmatrix}, \quad (2.6)$$

Based on the structure of the KKT system (2.5), Schur complement reduction can be applied to reduce the system (2.5) to a saddle point problem, the symmetric system can be presented as:

$$\begin{bmatrix} \tilde{H}_i & F^\top \\ F & 0 \end{bmatrix} \begin{bmatrix} \Delta y \\ \Delta \pi \end{bmatrix} = - \begin{bmatrix} \tilde{r}_{\text{stat}}^i \\ r_{\text{eq}}^i \end{bmatrix}, \quad (2.7)$$

with $\tilde{H}_i = H + G^\top T_i^{-1} \Lambda_i G \succeq H$ and $\tilde{r}_{\text{stat}}^i = r_{\text{stat}}^i + G^\top T_i^{-1} (\Lambda_i r_{\text{ineq}}^i - r_{\text{comp}}^i)$. Notably, the additional term in the Hessian $G^\top T_i^{-1} \Lambda_i G$ can be regarded as the curve contribution from active constraints, which is positive semidefinite. This term can already be interpreted as a kind of regularization in the problems.

In the (2.7), assuming F has full row rank, the null space method can be sequentially applied to reduce the system. With Y, Z spanning the range and null space of F , respectively, the primal step can be decomposed as $\Delta y = Y \Delta y_y + Z \Delta y_z$. This yields the Schur-complement based reduced-Hessian system:

$$\tilde{M}_i \Delta y_z = - (Z^\top \tilde{r}_{\text{stat}}^i + Z^\top \tilde{H}_i Y \Delta y_y), \quad (2.8)$$

where the equality Jacobian F is eliminated by $Z^\top$ (with $Z^\top F^\top = 0$), and feasibility fixes the range-space component $\Delta y_y = -(FY)^{-1} r_{\text{eq}}^i$, the complemented reduced Hessian $\tilde{M}_i = Z^\top \tilde{H}_i Z$ and the right-hand side $Z^\top \tilde{r}_{\text{stat}}^i + Z^\top \tilde{H}_i Y \Delta y_y$. This formulation exposes the connection between the reduced Hessian $\tilde{M}_i$ and the original Hessian $\tilde{H}$.

2.2 Efficient Solution with Riccati Recursion

Within the context of optimal control problems, the KKT system (2.7) exhibits a tridiagonal structure due to the temporal nature of the problem, as:

$$\begin{bmatrix} \ddots & & & & & \\ & 0 & -I & & & \\ & -I \begin{pmatrix} \tilde{Q}_k & \tilde{S}_k^\top \end{pmatrix} A_k^\top & & & & \\ & 0 \begin{pmatrix} \tilde{S}_k & \tilde{R}_k \end{pmatrix} B_k^\top & & & & \\ & 0 & A_k & B_k & 0 & -I \\ & & & & -I & \ddots \end{bmatrix} \begin{bmatrix} \vdots \\ \Delta \pi_k \\ \Delta x_k \\ \Delta u_k \\ \Delta \pi_{k+1} \\ \vdots \end{bmatrix} = - \begin{bmatrix} \vdots \\ r_{\text{eq}}^{k-1} \\ \tilde{r}_{\text{stat},x}^k \\ \tilde{r}_{\text{stat},u}^k \\ r_{\text{eq}}^k \\ \vdots \end{bmatrix}, \quad (2.9)$$

where A_k and B_k denote the Jacobians of the equality constraints with respect to the state and control variables at time step k , respectively. And the block matrices $\tilde{Q}_k$, $\tilde{R}_k$, and $\tilde{S}_k$ are defined as:

$$\begin{aligned}\tilde{Q}_k &= Q_k + (G_x^k)^\top \Lambda_k T_k^{-1} G_x^k, \\ \tilde{R}_k &= R_k + (G_u^k)^\top \Lambda_k T_k^{-1} G_u^k, \\ \tilde{S}_k &= S_k + (G_x^k)^\top \Lambda_k T_k^{-1} G_u^k,\end{aligned}$$

where Q_k , R_k , and S_k are the original block components constituting the Hessian matrix H_k , stands for state, control, and cross terms, respectively. G_x^k and G_u^k denote the corresponding Jacobians for the inequality constraints and possible box constraints on state and control.

This tridiagonal structure allows for efficient solution of the KKT system using Riccati recursion [3], which is consisted of a backward and forward pass. The backward pass computes the feedback gains K and cost-to-go matrices P , while the forward pass computes the optimal state and control trajectories, the algorithmic details of the Riccati recursion can be summarized as follows:

Algorithm 1 Riccati Recursion for Solving OCP Structure QP

```

1: Initialize:  $P_N \leftarrow \tilde{Q}_N$ ,  $p_N \leftarrow \tilde{r}_{\text{stat},x}^N$ 

    // Backward recursion
2: for  $k = N - 1$  down to  $0$  do
3:    $\tilde{M}_k \leftarrow \tilde{R}_k + B_k^\top P_{k+1} B_k$ 
4:    $\Gamma_k \leftarrow \tilde{S}_k + B_k^\top P_{k+1} A_k$ 
5:    $K_k \leftarrow -\tilde{M}_k^{-1} \Gamma_k$ 
6:    $k_k \leftarrow -\tilde{M}_k^{-1} \left( \tilde{r}_{\text{stat},u}^k + B_k^\top (P_{k+1} r_{\text{eq}}^k + p_{k+1}) \right)$ 
7:    $P_k \leftarrow \tilde{Q}_k + A_k^\top P_{k+1} A_k - \Gamma_k^\top \tilde{M}_k^{-1} \Gamma_k$ 
8:    $p_k \leftarrow \tilde{r}_{\text{stat},x}^k + A_k^\top (P_{k+1} r_{\text{eq}}^k + p_{k+1}) + \Gamma_k^\top k_k$ 
9: end for

    // Forward recursion
10: for  $k = 0$  to  $N - 1$  do
11:    $\Delta u_k \leftarrow K_k \Delta x_k + k_k$ 
12:    $\Delta x_{k+1} \leftarrow A_k \Delta x_k + B_k \Delta u_k + r_{\text{eq}}^k$ 
13:    $\Delta \pi_{k+1} \leftarrow P_{k+1} \Delta x_{k+1} + p_{k+1}$ 
14:    $\Delta t_k \leftarrow -(G_x^k \Delta x_k + G_u^k \Delta u_k + r_{\text{ineq}}^k)$ 
15:    $\Delta \lambda_k \leftarrow T_k^{-1} (-r_{\text{comp}}^k - \Lambda_k \Delta t_k)$ 
16: end for

```

By applying the Riccati recursion, we implicitly solve the KKT system (2.8) without explicitly forming the reduced Hessian $\tilde{M}_i$, and the computational complexity is reduced from $\mathcal{O}(N^3)$ to $\mathcal{O}(N)$, where N is the horizon length.

2.3 Adaptive Barrier Update

In this section, two closely related adaptive barrier update strategies are discussed for the primal-dual IPM. While the monotone decrease of the barrier parameter τ , such as the Fiacco-McCormick scheme [7], enjoys strong convergence guarantees, it may lead to slow convergence and practical limitations [8]. We therefore prefer a more aggressive, adaptive update of τ to achieve faster convergence, which is the focus of this section

2.3.1 Mehrotra’s Predictor-Corrector

For interior–point method, the Mehrotra’s predictor–corrector scheme is usually considered as the default choice for fast convergence [9], which adaptively updates the barrier parameter τ and adds a second-order corrector for improving the direction, at essentially the cost of one extra back-solve. In the stratdety, the barrier parameter τ is defined as:

$$\tau_i = \sigma_i \mu_i, \quad \mu_i = \frac{t_i^\top \lambda_i}{n_c}, \quad (2.10)$$

where μ_i is the duality measure, n_c is the number of inequality constraints, and $\sigma_i \in [0, 1]$ is the centering parameter that controls the trade-off between centering and aggressive steps. And in each iteration, two solves of (2.5) that share the same left-hand side and differ only in the complementarity right-hand side are performed, which involves two steps: predictor step (affine scaling step), and corrector step.

Predictor Step. The strategy starts with computing a predictor step, the linear system (2.5) is solved with $\sigma_i = 0$ in the complementarity residual r_{comp}^i and no correction considered, i.e. the resulting complementarity residual is written as:

$$r_{\text{comp}}^{\text{aff}} = T_i \Lambda_i \mathbf{1} - \sigma_i \mu_i \mathbf{1} = T_i \Lambda_i \mathbf{1}, \quad (2.11)$$

to obtain the affine scaling direction $(\Delta y_{\text{aff}}, \Delta \pi_{\text{aff}}, \Delta \lambda_{\text{aff}}, \Delta t_{\text{aff}})$. The affine step is used to estimate the centering parameter σ_i for the next step.

Centering. After the predictor step is obtained, The centering parameter σ is updated based on the estimated duality measure μ_{aff} after the affine step, which is defined as:

$$\sigma_i = (\mu_{\text{aff}}/\mu_i)^3 \quad (2.12)$$

$$\mu_{\text{aff}} = \frac{(t_i + \alpha_{\text{prim}}^{\text{aff}} \Delta t_{\text{aff}})^\top (\lambda_i + \alpha_{\text{dual}}^{\text{aff}} \Delta \lambda_{\text{aff}})}{n_c}, \quad (2.13)$$

where the step size $\alpha_{\text{prim}}^{\text{aff}}$ and $\alpha_{\text{dual}}^{\text{aff}}$ are determined by ensuring that the updated slack variables and Lagrange multipliers remain positive, i.e.

$$\alpha_{\text{prim}}^{\text{aff}} = \min \left(1, \min_{j: \Delta t_{\text{aff},j} < 0} -\frac{t_j^i}{\Delta t_{\text{aff},j}} \right), \quad (2.14)$$

$$\alpha_{\text{dual}}^{\text{aff}} = \min \left(1, \min_{j: \Delta \lambda_{\text{aff},j} < 0} -\frac{\lambda_j^i}{\Delta \lambda_{\text{aff},j}} \right). \quad (2.15)$$

If $\mu_{\text{aff}} \ll \mu_i$, the affine step achieves a substantial reduction in the duality gap, which sharply gives $\sigma_i \rightarrow 0$ (near-Newton, aggressive), while a stalled step ($\mu_{\text{aff}} \approx \mu_i$) gives $\sigma_i \rightarrow 1$ (recentering toward the central path).

Corrector Step. With the updated centering parameter σ_i and calculated affine steps, the corrector step is computed by solving the linear system (2.5) again. In the corrector step, the complementarity residual is modified to include a second-order correction term that accounts for the nonlinear effects of the affine step, which is given by:

$$r_{\text{comp}}^{\text{cc}} = T_i \Lambda_i \mathbf{1} + \Delta T_{\text{aff}} \Delta \Lambda_{\text{aff}} \mathbf{1} - \sigma_i \mu_i \mathbf{1}, \quad (2.16)$$

The term $\Delta T_{\text{aff}} \Delta \Lambda_{\text{aff}} \mathbf{1}$ cancels the second-order error that the affine step leaves in $t_j \lambda_j$, and $-\sigma_i \mu_i \mathbf{1}$ supplies the centering.

This second solve returns the final direction $(\Delta y, \Delta \pi, \Delta \lambda, \Delta t)$ directly, a step size control will be followed to ensure the positivity of the slack variables and Lagrange multipliers.

In practice, the corrector step is usually applied conditionally, the mechanism is, therefore, called conditional Mehrotra's predictor-corrector step [8], where the

corrector step is only applied if the affine step does not largely increase the estimated duality measure, i.e., $\mu_{\text{aff}} \leq 2\mu_i$.

2.3.2 Mehrotra’s Probing

However, Mehrotra’s predictor-corrector scheme is not always the best choice, particularly in the nonconvex case. Much as SQP methods form and use a local Hessian at each step, the nonconvex QP requires regularizing its possibly indefinite Hessian at every iterate to obtain a valid descent direction. The second-order correction in corrector step (2.16) may not be able to ensure a descent direction in the nonconvex case.

As mentioned in [8], the Mehrotra’s probing strategy can be applied alternatively to the predictor-corrector scheme, which taking a more conservative approach to update the barrier parameter τ by discarding the corrector step in predictor-corrector scheme but keeps the centering step. The corresponding complementarity residual is given by:

$$r_{\text{comp}}^{\text{probe}} = T_i \Lambda_i \mathbf{1} - \sigma_i \mu_i \mathbf{1}, \quad (2.17)$$

where the centering parameter σ_i is updated using (2.12) after the affine step, as in the predictor-corrector scheme.

2.4 Step Size Control

In this section, we discuss the step size control employed in the primal-dual interior point solver. The fraction-to-the-boundary (FTB) rule is first introduced as a common strategy for step size control in IPM, which ensures that the updated slack variables and Lagrange multipliers remain positive. Then, a two-phase line search with backtracking is proposed to ensure progress in either the objective function or the constraint violation, particularly in nonconvex cases where the FTB rule may not be sufficient to guarantee convergence.

2.4.1 Fraction-to-the-Boundary Rule

The fraction-to-the-boundary (FTB) rule is a common strategy for step size control in primal-dual interior point methods. The maximal step size $\alpha_{\text{prim},\text{max}}^i$ and $\alpha_{\text{dual},\text{max}}^i$ are computed without violating the positivity of the slack variables

and Lagrange multipliers, respectively, as:

$$\alpha_{\text{prim},\max}^i = \min_{j:\Delta t_j < 0} -\frac{t_j^i}{\Delta t_j}, \quad (2.18)$$

$$\alpha_{\text{dual},\max}^i = \min_{j:\Delta \lambda_j < 0} -\frac{\lambda_j^i}{\Delta \lambda_j}. \quad (2.19)$$

Then, in practice, the actual step sizes are chosen as a fraction of the maximum step sizes, i.e.,

$$\alpha_{\text{prim}}^i = \min(1, \eta \alpha_{\text{prim},\max}^i), \quad (2.20)$$

$$\alpha_{\text{dual}}^i = \min(1, \eta \alpha_{\text{dual},\max}^i), \quad (2.21)$$

where $\eta \in (0, 1)$ is a parameter that controls the fraction of the maximum step size to be taken, typically set to a value close to 1 to ensure sufficient progress while maintaining feasibility.

2.4.2 Two-Phase Line Search with Backtracking

In nonconvex cases, the FTB rule may not be sufficient to guarantee convergence, as the search direction with FTB-controlled step size may not descend.

A two-phase line search inspired by the funnel method [10] is applied on top of the FTB step size. Denoting by $\theta = \|c\|$ the constraint violation at the current iterate and by $\rho = \nabla f^\top d$ the directional derivative of the objective along the search direction d , the *switching condition*:

$$\alpha_{\text{prim}}(-\rho) > \kappa_s \theta^2, \quad \kappa_s > 0, \quad (2.22)$$

decides which of the two phases the trial step belongs to. Because (2.22) involves only quantities of the current iterate and the trial step size, it is evaluated *before* a candidate is formed.

If (2.22) fails, the predicted objective decrease is small relative to the constraint violation, so objective descent is not yet a meaningful measure of progress and the iterate is in the *h-phase*. The full FTB step is then accepted without any merit evaluation. This is justified by the structure of the subproblem: the constraints of the QP are affine and the Newton direction solves the corresponding rows of the linear system exactly, so $\theta(\alpha_{\text{prim}}) = (1 - \alpha_{\text{prim}}) \theta$ holds exactly along the step. The constraint violation therefore decreases monotonically in α_{prim} , and

the largest admissible step gives the fastest feasibility progress.

If (2.22) holds, sufficient objective progress is expected and the iterate is in the *f-phase*. A candidate iterate $z^+(\alpha_{\text{prim}})$ is formed in the primal variables (x, u, s) , its objective f^+ is evaluated, and the step is accepted when the Armijo condition [11]

$$f^+ \leq f + \gamma \alpha_{\text{prim}} \rho, \quad \gamma \in (0, 1), \quad (2.23)$$

is satisfied. Otherwise the step size is contracted, $\alpha_{\text{prim}} \leftarrow \beta \alpha_{\text{prim}}$ with $\beta \in (0, 1)$, and the search repeats. Contraction shrinks the left-hand side of (2.22), so a trial that starts in the f-phase may switch to the h-phase after a few backtracks and terminate there. If all n_{max} trials are rejected, the search reverts to the original FTB step size. The dual step size α_{dual} is left untouched throughout, as the merit function reads only the primal variables. The procedure is summarized in Algorithm 2.

Algorithm 2 Two-phase line search

```

1: Input: iterate  $z$ , direction  $d$ , FTB steps  $\alpha_{\text{prim}}, \alpha_{\text{dual}}$ ,  $f$ ,  $\theta = \|c\|$ ,  $\rho = \nabla f^\top d$ 
2:  $\alpha_{\text{prim}}^{\text{FTB}} \leftarrow \alpha_{\text{prim}}$ 
3: for  $\text{ls} = 1$  to  $n_{\text{max}}$  do
4:   if  $\alpha_{\text{prim}}(-\rho) \leq \kappa_s \theta^2$  then                                 $\triangleright$  h-phase: feasibility dominates
5:     break                                                             $\triangleright$  accept FTB-scaled step
6:   end if
7:   form candidate  $z^+(\alpha_{\text{prim}})$ 
8:   compute  $f^+$ 
9:   if  $f^+ \leq f + \gamma \alpha_{\text{prim}} \rho$  then                                 $\triangleright$  f-phase: Armijo
10:    break
11:  end if
12:  if  $\text{ls} = n_{\text{max}}$  then
13:     $\alpha_{\text{prim}} \leftarrow \alpha_{\text{prim}}^{\text{FTB}}$                                  $\triangleright$  all trials rejected: revert to the FTB step
14:  else
15:     $\alpha_{\text{prim}} \leftarrow \beta \alpha_{\text{prim}}$ 
16:  end if
17: end for
18: return  $\alpha_{\text{prim}}, \alpha_{\text{dual}}$ 

```

Two simplifications distinguish this scheme from the funnel method of [10]. First, no funnel bound θ_{max} is maintained. In the funnel method, a trial iterate is admissible only if it stays inside the relaxed feasible set $\{z : \theta(z) \leq \theta_{\text{max}}\}$, and the bound is contracted whenever a feasibility step is accepted. Here the constraints are affine, so $\theta(\alpha_{\text{prim}}) = (1 - \alpha_{\text{prim}})\theta \leq \theta$ for every $\alpha_{\text{prim}} \in (0, 1]$: no trial step

can increase the constraint violation, the funnel membership test is satisfied by construction and carries no information, and the h-phase acceptance test and the funnel update become redundant.

Second, the switching condition (2.22) is built on the first-order term $-\nabla f^\top d$ alone, in line with the Armijo condition [11], whereas [10] uses the second-order model reduction $\Delta m_f = -\frac{1}{2}d^\top H d - \nabla f^\top d$. The reason is the nonconvexity of the subproblem: since H is generally indefinite, the sign of $d^\top H d$ is not controlled, so the second-order contribution to Δm_f may be misleading rather than informative for classifying the step. To compensate for dropping the second-order term, the constant κ_s is set to a small value, so that objective-reducing steps are not misclassified as h-phase steps.

Chapter 3

Primal Regularization for Indefinite Quadratic Programs

In this chapter, we turn our attention to QPs with an indefinite Hessian matrix, namely indefinite QPs. Section 3.1 classifies the existing primal regularization approaches by the matrix they target. Section 3.2 then develops the concept of Riccati-based primal regularization, which applies the correction stagewise inside the Riccati recursion. Section ?? analyses why the minimum-perturbation rule fails in this setting, and derives an improved stage regularization strategy from that analysis. Section 3.3 concludes the chapter by proving that the proposed regularization becomes asymptotically inactive as the iterates approach the solution.

3.1 Primal regularization with Riccati-based Detection

In indefinite QPs, the regularization needs to be applied to the Hessian matrix to ensure a descent search direction in the interior point method. The resulting regularized reduced KKT system can be generally presented as:

$$\begin{bmatrix} \tilde{H}_i + E_i & F^\top \\ F & 0 \end{bmatrix} \begin{bmatrix} \Delta y \\ \Delta \pi \end{bmatrix} = - \begin{bmatrix} \tilde{r}_{\text{stat}}^i \\ r_{\text{eq}}^i \end{bmatrix}, \quad (3.1)$$

where E_i is the adjustment matrix that renders the Hessian positive definite at the i -th iteration. This process is referred to as primal regularization.

Two choices are left open by this formulation: how the adjustment E_i is computed, and which matrix it is derived from and applied to. The two are

independent, and the first is common ground, since the same numerical strategies apply whichever matrix is targeted. Section 3.1.1 therefore introduces these strategies first, after which Section 3.1.2 and Section 3.1.3 distinguish the two types of primal regularization by their target matrix, the full Hessian and the reduced Hessian.

3.1.1 Regularization Strategies

Three families of strategies are commonly used to construct the adjustment matrix E_i [12–14]:

- Eigenvalue modification (EM): replaces the negative eigenvalues of $\tilde{H}_i$ through a spectral decomposition.
- Modified Cholesky factorization (MC): corrects the inertia of $\tilde{H}_i$ by adjusting the diagonal pivots during factorization.
- Levenberg–Marquardt (LM): adds a scaled identity matrix to $\tilde{H}_i$ until it becomes positive definite.

The families differ in how the correction is obtained, and therefore in cost: EM requires a full spectral decomposition, MC adjusts the pivots during the factorization itself, and LM applies a uniform shift without any decomposition at all. From these families we select four methods for this work: PROJECT and MIRROR from EM [15], GMW from MC [16], and inertia correction from LM [17].

Assume a matrix A admits the eigenvalue decomposition $A = VD_EV^\top$ and the Cholesky factorization $A = LD_IL^\top$, where V is an orthogonal matrix of eigenvectors, L is a unit lower triangular matrix, D_E is a diagonal matrix of real eigenvalues, and D_I is a diagonal matrix whose signs give the inertia of A . In this notation, the four methods read as follows:

- **PROJECT** (Minimal eigenvalue modification):

$$\text{project}(A, \epsilon) := V \max(\epsilon I, D_E) V^\top.$$

Where the max operator is applied component-wise. This approach clips any negative or near-zero eigenvalues to a small positive constant $\epsilon > 0$, which keeps a minimal adjustment to the original matrix for positive definiteness, thus the minimal Frobenius norm of the adjustment matrix E [12], but may lead to an increase in the condition number if the original eigenvalues were not close to zero.

- **MIRROR** (Absolute eigenvalue modification):

$$\text{mirror}(A, \epsilon) := V \max(\epsilon I, |D_E|) V^\top.$$

This method reflects negative eigenvalues to their positive counterparts. It preserves the magnitude of the curvature in all directions, which often provides better scaling, but may result in a large modification to the original matrix.

- **GMW** (Modified Cholesky factorization):

$$\text{gmw}(A, \epsilon) := L D_I L^\top, \quad D_I \succeq \epsilon I.$$

Instead of a spectral decomposition, the factorization is carried out in place, and each diagonal pivot is raised, whenever necessary, to keep it above ϵ and to bound the magnitude of the corresponding column of L [16]. Bounding the pivots from below therefore also bounds the condition number of the factor, and the accumulated increments form a nonnegative diagonal correction E with $A + E = L D_I L^\top$.

This avoids the cost of an eigendecomposition and still guarantees positive definiteness, but the resulting E is generally not minimal and depends on the variable ordering, so it tends to over-modify A compared with the eigenvalue-based methods.

- **Inertia Correction** (Levenberg–Marquardt):

$$\text{ic}(A, \Delta) := A + \Delta I, \quad \Delta \geq 0.$$

A uniform multiple of the identity is added, so every eigenvalue is shifted by the same amount, $\lambda_k(A + \Delta I) = \lambda_k(A) + \Delta$. In practice $\lambda_{\min}$ is not computed directly, and Δ is found by repeatedly attempting the $\text{LDL}^\top$ factorization and increasing Δ until the inertia is correct, i.e. until every diagonal entry of D_I is positive. The shift leaves the eigenvectors unchanged and is very cheap and robust, however, because it acts equally in all directions, it also inflates the already-positive eigenvalues and can degrade the scaling when only a few directions are indefinite.

3.1.2 Full Hessian regularization

The full hessian regularization take the full Hessian matrix $\tilde{H}$ as the target matrix. Commonly, in the optimal control structure, the full Hessian is regularized stage-wisely by operating on the block Hessian matrix $\tilde{H}_k$ at k stage,

$$\tilde{H}_k^{\text{reg}} = \begin{bmatrix} \tilde{Q}_k & \tilde{S}_k^\top \\ \tilde{S}_k & \tilde{R}_k \end{bmatrix} + E_k \succ 0, \quad k = 0, 1, \dots, N-1, \quad (3.2)$$

where E_k could be a diagonal matrix or a full matrix, depending on the regularization strategy mentioned in 3.1.1.

The full Hessian regularization is direct and usually simple to implement, but it can be too conservative without considering the definiteness of the reduced Hessian $\tilde{M}$, thus slow down the convergence.

3.1.3 Reduced Hessian regularization

The reduced Hessian regularization take the reduced Hessian matrix $\tilde{M}$ as the target matrix. According to the Theorem 1, which is mentioned in [15] (for proof, see [12]):

Theorem 1. *Assume that LICQ holds. The QP problem has a unique global optimum if and only if the reduced Hessian is positive definite:*

$$M = Z^\top H Z \succ 0. \quad (3.3)$$

The positive definiteness of the reduced Hessian is a necessary and sufficient condition for the QP admitting a unique global optimum. It is thus reasonable to derive the correction from the reduced Hessian $\tilde{M}$ instead of full Hessian $\tilde{H}$, which yields a less conservative regularization that still effective. However, from definition, computing the reduced Hessian directly requires a null-space basis of the equality constraints, which can be achieved via condensing in optimal control problems [18], this step generally introduces extra computational overhead.

We thus propose a Riccati-based primal regularization method, which naturally obtains the reduced Hessian $\tilde{M}$ implicitly via Riccati recursion, and regularizes the reduced Hessian $\tilde{M}$ stage-wisely during the Riccati recursion to achieve a positive definite reduced Hessian $\tilde{M}$.

3.1.4 Block Reduced Hessian From Riccati Recursion

As mentioned in [19], the Riccati recursion can be interpreted as a block factorization of the KKT system in (2.7). In this subsection, we will further show that the Riccati recursion can be regarded as the block $\text{LDL}^\top$ factorization of reduced Hessian $\tilde{M}_i$ in (2.8), which justifies the stage regularization on block reduced Hessian.

From the perspective of the Riccati recursion, the backward pass can be concluded as follows:

Algorithm 3 Backward Pass of Riccati Recursion

- 1: **Initialize:** $P_N \leftarrow \tilde{Q}_N$, $p_N \leftarrow \tilde{r}_{\text{stat},x}^N$
 - 2: **for** $k = N - 1$ **down to** 0 **do**
 - 3: $\tilde{M}_k \leftarrow \tilde{R}_k + B_k^\top P_{k+1} B_k$
 - 4: $\Gamma_k \leftarrow \tilde{S}_k + B_k^\top P_{k+1} A_k$
 - 5: $K_k \leftarrow -\tilde{M}_k^{-1} \Gamma_k$
 - 6: $k_k \leftarrow -\tilde{M}_k^{-1} \left(\tilde{r}_{\text{stat},u}^k + B_k^\top (P_{k+1} r_{\text{eq}}^k + p_{k+1}) \right)$
 - 7: $P_k \leftarrow \tilde{Q}_k + A_k^\top P_{k+1} A_k + \Gamma_k^\top K_k$
 - 8: $p_k \leftarrow \tilde{r}_{\text{stat},x}^k + A_k^\top (P_{k+1} r_{\text{eq}}^k + p_{k+1}) + \Gamma_k^\top k_k$
 - 9: **end for**
-

We can find that $\tilde{M}_k = \tilde{R}_k + B_k^\top P_{k+1} B_k$ is exactly the block reduced Hessian obtained by Riccati recursion. In practice the inverse of $\tilde{M}_k$ is not computed directly; instead, the Cholesky factorization of $\tilde{M}_k$ is computed for efficiency.

From the perspective of the reduced Hessian $\tilde{M}_i$ in (2.8), eliminating the states through the dynamics leaves a dense reduced Hessian $\tilde{M}_i$ in the inputs $u_0, \dots, u_{N-1}$, whose density stems from the coupling of any two stages via the state transition $\Phi_{i,j}$, defined as:

$$\Phi_{i,j} = \begin{cases} \prod_{k=j}^{i-1} A_k = A_{i-1} A_{i-2} \cdots A_j, & \text{if } i > j, \\ I, & \text{if } i = j. \end{cases}$$

The backward pass in Algorithm 1 is exactly a block $\text{LDL}^\top$ factorization of this $\tilde{M}_i$, carried out without ever forming it explicitly. The connection is made through the Schur complement: each cost-to-go matrix P_k is the Schur complement obtained by eliminating the states of the later stages, so that the pivot block $\tilde{M}_k$ is precisely the Schur complement of stage k in $\tilde{M}_i$ after all later stages have been eliminated. Consequently, the recursion $P_k = \tilde{Q}_k + A_k^\top P_{k+1} A_k - \Gamma_k^\top \tilde{M}_k^{-1} \Gamma_k$

is nothing but the successive formation of these Schur complements.

Collecting the pivots $\tilde{M}_{N-1}, \dots, \tilde{M}_0$ into the block-diagonal factor D and eliminating the stages in the same order as the backward pass, from stage $N - 1$ down to stage 0, the block LDL[⊤] factorization of the reduced Hessian $\tilde{M}_i$ can be expressed as:

$$\tilde{M}_i = L \begin{bmatrix} \tilde{M}_{N-1} & 0 & \cdots & 0 \\ 0 & \tilde{M}_{N-2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \tilde{M}_0 \end{bmatrix} L^\top$$

where the accumulated back-substitution coefficients, built from the feedback gains K_k and the transitions $\Phi_{i,j}$, form the unit lower-triangular factor L :

$$L = \begin{bmatrix} I & 0 & \cdots & 0 & 0 \\ -B_{N-2}^\top K_{N-1}^\top & I & \cdots & 0 & 0 \\ -B_{N-3}^\top \Phi_{N-1,N-2}^\top K_{N-1}^\top & -B_{N-3}^\top K_{N-2}^\top & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ -B_0^\top \Phi_{N-1,1}^\top K_{N-1}^\top & -B_0^\top \Phi_{N-2,1}^\top K_{N-2}^\top & \cdots & -B_0^\top K_1^\top & I \end{bmatrix}.$$

This establishes the connection between positive definiteness of the reduced Hessian $\tilde{M}_i$ and the positive definiteness of the block reduced Hessians $\tilde{M}_k$ obtained from Riccati recursion, which justifies the stage regularization on block reduced Hessian $\tilde{M}_k$.

3.2 Riccati-based Primal Regularization

In this section, we will introduce the idea and concept of Riccati-based primal regularization, a stage-wise regularization method for the reduced Hessian $\tilde{M}$ based on the Riccati recursion in two sense: first, Riccati recursion is used for detecting the indefiniteness of the reduced Hessian $\tilde{M}$, second, Riccati recursion is used for regularizing the reduced Hessian $\tilde{M}$ stage-wisely to ensure a positive definite reduced Hessian $\tilde{M}$.

3.2.1 Stage Regularization in Riccati Recursion

With this connection, we can regularize the reduced Hessian $\tilde{M}_i$ without explicitly computing it, but by stage-wisely applying the regularization to the block reduced Hessian $\tilde{M}_k^i$ during the Riccati recursion,

As mentioned in 3.1.4, the Cholesky factorization of $\tilde{M}_k^i$ is computed for efficiency, which requires the positive definiteness of $\tilde{M}_k^i$. This can be regarded as a detection of the indefiniteness of the reduced Hessian $\tilde{M}_i$. And when the Cholesky factorization fails, which indicates that the reduced Hessian $\tilde{M}_i$ is indefinite, we can regularize the reduced Hessian block $\tilde{M}_k^i$ by adding a block adjust matrix E_k^i , i.e. $\tilde{M}_k^i + E_k^i$. The E_k^i can be computed from the strategies mentioned in 3.1.1. This gives a modification of the backward pass of Riccati recursion and can be concluded as follows:

Algorithm 4 Backward Pass of Modified Riccati Recursion

```

1: Initialize:  $P_N \leftarrow \tilde{Q}_N$ ,  $p_N \leftarrow \tilde{r}_{\text{stat},x}^N$ ,  $\lambda > 0$ 

2: for  $k = N - 1$  down to 0 do
3:    $\tilde{M}_k \leftarrow \tilde{R}_k + B_k^\top P_{k+1} B_k$ 
4:    $L_k \leftarrow \text{Cholesky}(\tilde{M}_k)$ 
5:   if factorization fails then
6:      $\tilde{M}_k \leftarrow \tilde{M}_k + E_k$  ▷ Apply regularization
7:      $L_k \leftarrow \text{Cholesky}(\tilde{M}_k)$ 
8:   end if
9:    $\Gamma_k \leftarrow \tilde{S}_k + B_k^\top P_{k+1} A_k$ 

10:   $K_k \leftarrow -L_k^{-\top} L_k^{-1} \Gamma_k$ 
11:   $k_k \leftarrow -L_k^{-\top} L_k^{-1} \left( \tilde{r}_{\text{stat},u}^k + B_k^\top (P_{k+1} r_{\text{eq}}^k + p_{k+1}) \right)$ 

12:   $P_k \leftarrow \tilde{Q}_k + A_k^\top P_{k+1} A_k + \Gamma_k^\top K_k$ 
13:   $p_k \leftarrow \tilde{r}_{\text{stat},x}^k + A_k^\top (P_{k+1} r_{\text{eq}}^k + p_{k+1}) + \Gamma_k^\top k_k$ 
14: end for

```

Since the block reduced Hessians $\tilde{M}_k$ are exactly the pivots of the block $L^\top DL$ factorization of $\tilde{M}_i$, enforcing the positive definiteness of every $\tilde{M}_k$ during the backward pass is equivalent to regularizing the reduced Hessian $\tilde{M}_i$ itself, and thus recovers the SOSC in Theorem 1. The regularization is therefore applied only where it is needed while the structure-exploiting complexity of the Riccati recursion is preserved.

3.2.2 Failure of Minimum-Perturbation Regularization

In this subsection, we will show that the minimum perturbation strategy, i.e. the PROJECT [15], which is commonly the first choice for regularization with the least modification of the original problem, can cause condition number issue in the stage regularization by introducing a general example.

Suppose at the stage k , the reduced Hessian block $\tilde{M}_k$ is not positive definite. Without loss of generality, we can assume $\tilde{M}_k$ has only one negative eigenvalue λ_j , and the corresponding eigenvector is v_j . The minimum perturbation strategy is applied as:

$$E_k = (-\lambda_j + \epsilon)v_j v_j^\top, \quad (3.4)$$

where the ϵ is a small positive value to ensure the positive definiteness of the regularized reduced Hessian block.

By Sherman-Morrison formula, the inverse of the regularized reduced Hessian block can be expressed as:

$$\begin{aligned} (\tilde{M}_k + E_k)^{-1} &= \tilde{M}_k^{-1} + \frac{\lambda_j - \epsilon}{\epsilon \lambda_j} v_j v_j^\top \\ &= \tilde{M}_k^{-1} + \left(\frac{1}{\epsilon} - \frac{1}{\lambda_j}\right) v_j v_j^\top, \end{aligned}$$

and the update of cost-to-go matrix P_k follows as:

$$\begin{aligned} P_k &= \tilde{Q}_k + A_k^\top P_{k+1} A_k - \Gamma_k^\top K_k \\ &= \tilde{Q}_k + A_k^\top P_{k+1} A_k - \Gamma_k^\top \tilde{M}_k^{-1} \Gamma_k - \left(\frac{1}{\epsilon} - \frac{1}{\lambda_j}\right) \Gamma_k^\top v_j v_j^\top \Gamma_k \\ &= P_k^{\text{unreg}} - \left(\frac{1}{\epsilon} - \frac{1}{\lambda_j}\right) \Gamma_k^\top v_j v_j^\top \Gamma_k, \end{aligned} \quad (3.5)$$

note that this negative rank-1 modification can already render P_k itself indefinite, independently of its downstream effect on $\tilde{M}_{k-1}$, whenever the perturbation magnitude exceeds the smallest eigenvalue of P_k^{unreg} . This can happen even when the perturbation on $\tilde{M}_k$ is moderate, provided that the negative-curvature direction v_j is strongly coupled to the state through Γ_k .

Consequently, the reduced Hessian block $\tilde{M}_{k-1}$ at the previous stage as:

$$\tilde{M}_{k-1} = \tilde{M}_{k-1}^{\text{unreg}} - \left(\frac{1}{\epsilon} - \frac{1}{\lambda_j}\right) z_j z_j^\top, \quad (3.6)$$

where $z_j = B_{k-1}^\top \Gamma_k^\top v_j$ is a column vector.

And the condition number of the regularized reduced Hessian block $\tilde{M}_{k-1}$ can be approximated as:

$$\kappa(\tilde{M}_{k-1}) \approx \frac{c(\epsilon + |\lambda_j|)}{\epsilon|\lambda_j|}, \quad (3.7)$$

where $c = \frac{\|z_j\|^2}{\lambda_{\min}(\tilde{M}_{k-1}^{\text{unreg}})}$ as an $O(1)$ constant.

Notably, as the ϵ is a small positive value, the presence of the negative rank-1 update $-\frac{1}{\epsilon}z_j z_j^\top$ will dominate, not only severely deteriorates its condition number, but also likely to make $\tilde{M}_{k-1}$ indefinite, which can further cause a domino effect through the stages.

3.2.3 Insights and Proposed Methods

This issue reveals two insights in stage regularization:

- The minimum modification objective [20] is not the foremost consideration for stage regularization, as it can cause severe condition number issue in the preceding stages. A trade-off between perturbation norm and condition number must be carefully treated.
- The cost-to-go matrix P_k must be incorporated into the regularization design, as the severity of propagation depends on the coupling structure between stages, which a purely local scheme cannot capture.

From the first insight, instead of the minimum perturbation strategy, an intuitive, but effective, approach is to use the absolute value of the negative eigenvalues or inertia as the perturbation magnitude, which can avoid the severe condition number issue in the preceding stages. When it comes to the eigenvalue modification strategy, this is the MIRROR strategy [15], and when it comes to the modified Cholesky factorization strategy, this is the Type-II strategy [20], of which GMW is a common representative.

From the second insight, we can incorporate the cost-to-go matrix P_k into the strategy design, which also can be achieved by adding an adjustment matrix $E_{k+1,P}^i$ to the cost-to-go matrix P_k directly, so that the propagation of the negative curvature can be taken into account in the regularization design.

The resulting improved stage regularization strategy can be summarized as follows:

Algorithm 5 Backward Pass of Improved Modified Riccati Recursion

```

1: Initialize:  $P_N \leftarrow \tilde{Q}_N$ ,  $p_N \leftarrow \tilde{r}_{\text{stat},x}^N$ ,  $\mathcal{E}(\cdot)$ : MIRROR or Type-II strategy

2: for  $k = N - 1$  down to 0 do
3:    $P_{k+1}^{\text{reg}} \leftarrow P_{k+1}$  ▷ Default with no regularization
4:    $\tilde{M}_k \leftarrow \tilde{R}_k + B_k^\top P_{k+1}^{\text{reg}} B_k$ 
5:    $L_k \leftarrow \text{Cholesky}(\tilde{M}_k)$ 
6:   if factorization fails then
7:      $P_{k+1}^{\text{reg}} \leftarrow \mathcal{E}(P_{k+1})$  ▷ Regularize cost-to-go matrix
8:      $\tilde{M}_k \leftarrow \tilde{R}_k + B_k^\top P_{k+1}^{\text{reg}} B_k$ 
9:      $\tilde{M}_k^{\text{reg}} \leftarrow \mathcal{E}(\tilde{M}_k)$  ▷ Regularize reduced Hessian block
10:     $L_k \leftarrow \text{Cholesky}(\tilde{M}_k^{\text{reg}})$ 
11:   end if
12:    $\Gamma_k \leftarrow \tilde{S}_k + B_k^\top P_{k+1}^{\text{reg}} A_k$ 

13:    $K_k \leftarrow -L_k^{-\top} L_k^{-1} \Gamma_k$ 
14:    $k_k \leftarrow -L_k^{-\top} L_k^{-1} \left( \tilde{r}_{\text{stat},u}^k + B_k^\top (P_{k+1}^{\text{reg}} r_{\text{eq}}^k + p_{k+1}) \right)$ 

15:    $P_k \leftarrow \tilde{Q}_k + A_k^\top P_{k+1}^{\text{reg}} A_k + \Gamma_k^\top K_k$ 
16:    $p_k \leftarrow \tilde{r}_{\text{stat},x}^k + A_k^\top (P_{k+1}^{\text{reg}} r_{\text{eq}}^k + p_{k+1}) + \Gamma_k^\top k_k$ 
17: end for

```

In Algorithm 5, when the Cholesky factorization fails, the regularization acts at two levels. Applying $\mathcal{E}(\cdot)$ to the cost-to-go matrix proactively convexifies the term that enters the current stage, so that the negative-curvature propagation identified in the second insight is accounted for in $\tilde{M}_k$, Γ_k , and P_k consistently. Applying it again to the reduced Hessian block acts as a local safeguard, ensuring that the Cholesky factorization succeeds. Both corrections are produced by the same non-minimal strategy $\mathcal{E}(\cdot)$, MIRROR or Type-II, rather than by the minimum-modification rule, which keeps the condition number of the regularized blocks bounded, as motivated by the first insight. When no factorization failure occurs, the recursion reduces to the standard Riccati backward pass.

3.3 Asymptotic Inactivity of the Regularization

In this section, we will prove that the reduced Hessian regularization via Riccati recursion is asymptotically inactive, which means that the regularization will be inactive when the iterates are sufficiently close to the optimal solution (y^*, λ^*) , following a similar proof strategy to [21].

We consider the regularized KKT formulation, and apply the null space method to (2.5) first. The resulting formulation is given as:

$$\begin{bmatrix} W_i & Z^\top G^\top \\ GZ & -\Lambda_i^{-1} T_i \end{bmatrix} \begin{bmatrix} \Delta y_z \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} r_{\text{stat, redu}}^i \\ \tilde{r}_{\text{ineq, redu}}^i \end{bmatrix}, \quad (3.8)$$

where

$$\begin{aligned} W_i &= Z^\top HZ + E_i = M + E_i, \\ r_{\text{stat, redu}}^i &= Z^\top r_{\text{stat}}^i + Z^\top HY \Delta y_y, \\ \tilde{r}_{\text{ineq, redu}}^i &= r_{\text{ineq}}^i + GY \Delta y_y - \Lambda_i^{-1} r_{\text{comp}}^i. \end{aligned}$$

This formulation eliminates the equality constraints, recasting the optimal control structure into a general QP form, and yields a saddle-point system coupling the reduced primal variable Δy_z (with regularized reduced Hessian W_i) and the dual variable $\Delta \lambda$. Eliminating $\Delta \lambda$ by a further Schur complement recovers the complemented reduced Hessian of (2.8) as:

$$\tilde{W}_i = W_i + Z^\top G^\top T_i^{-1} \Lambda_i GZ = \tilde{M}_i + E_i.$$

We will make the following assumptions for the proof:

Assumption 1. (y^*, λ^*) is a local (or global) minimizer for (2.1).

Assumption 2. Each local minimizer (y^*, λ^*) for (2.1) is isolated.

Assumption 3. Linear independence constraint qualification (LICQ) holds at the minimizer (y^*, λ^*) .

Assumption 4. Strict complementarity holds at the minimizer (y^*, λ^*) .

Assumptions 1–3 imply the second order sufficient condition (SOSC) for local optimality, so we have the following Lemma 2:

Lemma 2. Under Assumptions 1–3, for all $v \in C(y^*, \lambda^*) \setminus \{0\}$, it holds that $v^\top H v > 0$.

Proof. With Assumption 1, the second order necessary condition (SONC) for local optimality holds, which states that for all $v \in C(y^*, \lambda^*)$, we have $v^\top H v \geq 0$.

We proceed by contradiction. Suppose there exists $v \in C(y^*, \lambda^*) \setminus \{0\}$ such that $v^\top H v = 0$. Consider the point $y(\alpha) = y^* + \alpha v$ for $\alpha > 0$. Because the

constraints are linear and v is in the critical cone, $y(\alpha)$ remains feasible for sufficiently small α .

Since the objective function of (2.1) is quadratic, expanding $f(y(\alpha))$ yields:

$$f(y(\alpha)) = f(y^*) + \alpha \nabla f(y^*)^\top v + \frac{1}{2} \alpha^2 v^\top H v$$

As $\nabla f(y^*)^\top v = 0$, given the assumption that $v^\top H v = 0$, the equation reduces exactly to $f(y(\alpha)) = f(y^*)$. This implies that $y(\alpha)$ is also a local minimizer, which directly contradicts Assumption 2. Therefore, $v^\top H v > 0$ must hold. $\square$

Proposition 3. *Under Assumptions 1–4, given the sequence $\{\tilde{W}_i\}$ constructed by the Riccati-based regularization, as $\{(y_i, \lambda_i)\} \rightarrow (y^*, \lambda^*)$, $\tilde{W}_i = \tilde{M}_i = Z^\top \tilde{H}_i Z \succ 0$ for all i sufficiently large.*

Proof. Under Assumption 4, the critical cone $C(y^*, \lambda^*)$ reduces to the tangent space of the active constraints. Therefore, we have $C(y^*, \lambda^*) = \{v \mid Fv = 0, G_{\mathcal{A}}v = 0\}$.

For any v with $\|v\| = 1$, write $d = Zv \in \{d \mid Fd = 0\} \setminus \{0\}$ and let G_j denote the j -th row of G . The corresponding quadratic form of the reduced Hessian reads:

$$\begin{aligned} v^\top \tilde{M}_i v &= d^\top (H + G^\top T_i^{-1} \Lambda_i G) d \\ &= d^\top H d + \sum_j \frac{\lambda_j^i}{t_j^i} (G_j d)^2 \\ &= d^\top H d + \sum_{j \in \mathcal{A}} \frac{\lambda_j^i}{t_j^i} (G_j d)^2 + \sum_{j \notin \mathcal{A}} \frac{\lambda_j^i}{t_j^i} (G_j d)^2 \end{aligned}$$

We now proceed by contradiction. Assume there exists a sequence v_i with $\|v_i\| = 1$ and, passing to a subsequence if necessary, $v_i \rightarrow \bar{v}$ such that $v_i^\top \tilde{M}_i v_i \leq 0$ for all i . Writing $d_i = Zv_i$ and $\bar{d} = Z\bar{v}$, so that $d_i \rightarrow \bar{d}$, we can distinguish two cases:

For $G_{\mathcal{A}}\bar{d} = 0$, the limit direction satisfies $\bar{d} \in C(y^*, \lambda^*) \setminus \{0\}$, so $\bar{d}^\top H \bar{d} > 0$ follows by Lemma 2. Since the remaining terms are non-negative, $v_i^\top \tilde{M}_i v_i \geq d_i^\top H d_i$ with $d_i^\top H d_i \rightarrow \bar{d}^\top H \bar{d} > 0$. Hence $v_i^\top \tilde{M}_i v_i > 0$ for all sufficiently large i , contradicting the assumption.

For $G_{\mathcal{A}}\bar{d} \neq 0$, Assumption 4 gives $\lambda_j^i \rightarrow \lambda_j^* > 0$ and $t_j^i \rightarrow 0$ for $j \in \mathcal{A}$, so the term $\sum_{j \in \mathcal{A}} \frac{\lambda_j^i}{t_j^i} (G_j d_i)^2 \rightarrow \infty$, which dominates the other two terms, and thus $v_i^\top \tilde{M}_i v_i \rightarrow \infty$, contradicting the assumption.

Therefore, we can conclude that $\tilde{W}_i = \tilde{M}_i = Z^\top \tilde{H}_i Z \succ 0$ for all i sufficiently large, i.e., $E_i = 0$. $\square$

Proposition 3 shows the regularization is asymptotically inactive: once the iterates are close to (y^*, λ^*) , $E_i = 0$ and the regularized recursion coincides with the unregularized one. The solver thus recovers the exact Newton step and inherits the local convergence rate of the underlying interior-point method, without the regularization affecting its asymptotic behavior.

Chapter 4

A Benchmark for Indefinite OCP Quadratic Programs

In order to evaluate the performance of the proposed Riccati-based primal regularization, especially in the context of the subproblems of SQP in NMPC. In this chapter, we aim at constructing a benchmark of indefinite OCP QPs that arise in the SQP iterations of NMPC problems. We will first introduce the construction of the QP benchmark, including the formulation of QPs in benchmark, the OCP test problems and the strategy for collecting indefinite QPs in Section 4.1. Then we will analyze the collected problem instances to gain a better understanding of characteristics regarding definiteness in the indefinite OCP QP benchmark in Section 4.2.

4.1 Benchmark Construction

In this section, we will introduce the construction of the indefinite QP benchmark. We will first introduce the formulation of QPs in benchmark, which originate from the SQP iterations of the OCPs, and then we will introduce the eight OCP test problems used to construct the benchmark, which are formulated as fixed-time regulator problems with quadratic stage costs and box constraints on the controls. Then the collection strategy is designed to ensure that the collected QPs are representative of the indefinite QPs that arise in practice, and that they share the mathematical structure of the OCPs.

4.1.1 Formulation of QP subproblems from OCPs

We will start from the formulation of the OCPs to obtain the resulting QP subproblems that arise in the SQP iterations. In the benchmark, all optimal control problems share the same structure: a fixed-horizon, direct multiple-shooting [1] optimal control problem with a quadratic stage cost that penalizes deviations from given state and control references. The mathematical formulation of the OCPs is given as follows:

$$\begin{aligned}
\min_{\substack{x_0, \dots, x_N \\ u_0, \dots, u_{N-1}}} \quad & \sum_{k=0}^{N-1} \frac{1}{2} \begin{bmatrix} x_k - x_k^{\text{ref}} \\ u_k - u_k^{\text{ref}} \end{bmatrix}^\top \begin{bmatrix} \mathcal{Q} & \mathcal{S}_k^\top \\ \mathcal{S}_k & \mathcal{R} \end{bmatrix} \begin{bmatrix} x_k - x_k^{\text{ref}} \\ u_k - u_k^{\text{ref}} \end{bmatrix} + \frac{1}{2} \|x_N - x_N^{\text{ref}}\|_{\mathcal{Q}_N}^2 \\
\text{s.t.} \quad & r(x_0, x_N) = 0, \\
& x_{k+1} = f_k(x_k, u_k), \quad k = 0, \dots, N-1, \\
& \underline{g} \leq \mathcal{C}_k x_k + \mathcal{D}_k u_k \leq \bar{g}, \quad k = 0, \dots, N-1, \\
& \underline{h} \leq h_k(x_k, u_k) \leq \bar{h}, \quad k = 0, \dots, N-1,
\end{aligned} \tag{4.1}$$

where $x_{k+1} = f_k(x_k, u_k)$ is one integration step of the continuous dynamics over $[t_k, t_{k+1}]$ for notational simplicity, $r(x_0, x_N) = 0$ represents the initial and terminal equality constraints, $\mathcal{Q}, \mathcal{R}, \mathcal{Q}_N \succeq 0$ are weighting matrices, $\mathcal{S}_k$ is the state-control cost-coupling matrix, and $x_k^{\text{ref}}, u_k^{\text{ref}}$ are the reference points. $\mathcal{C}_k x_k + \mathcal{D}_k u_k$ includes the affine and box path constraints, h_k collects the nonlinear path constraints.

When using the exact Hessian of the Lagrangian in the SQP iterations, the resulting Hessian of the QP subproblem at stage $k = 0, \dots, N-1$ reads:

$$\begin{aligned}
H_k &= \nabla^2 \mathcal{L}(x, u, \pi, \lambda) \\
&= \begin{bmatrix} \mathcal{Q}_k & \mathcal{S}_k^\top \\ \mathcal{S}_k & \mathcal{R}_k \end{bmatrix} + \sum_{j=1}^{n_x} \pi_k \nabla^2 f_k(x_k, u_k) + \sum_{j=1}^{n_c} \lambda_k \nabla^2 h_k(x_k, u_k),
\end{aligned} \tag{4.2}$$

where n_x is the dimension of states and n_c is the number of nonlinear constraints, π_k and λ_k are the Lagrange multipliers associated with the dynamics and nonlinear constraints, respectively.

The stage cost is quadratic, so its Hessian $\begin{bmatrix} \mathcal{Q}_k & \mathcal{S}_k^\top \\ \mathcal{S}_k & \mathcal{R}_k \end{bmatrix}$ is constant and exact. The indefiniteness of H_k thus has two sources: a constant contribution from the cost block, and an iterate-dependent contribution from the two curvature terms, which is usually the dominant one during the SQP iterations. In these curvature

terms, each component Hessian $\nabla^2[f_k]_j$ and $\nabla^2[h_k]_j$ is symmetric but carries no definiteness guarantee, and the associated multipliers $(\pi_k)_j$ and $(\lambda_k)_j$ may take arbitrary sign and magnitude during the SQP iterations. Their weighted sum is thus a general symmetric and typically indefinite matrix. Adding it to the constant cost block can introduce negative eigenvalues in H_k , in particular when the multipliers are large, i.e. when constraints are strongly active or the current iterate is still far from a solution. Consequently, the exact-Hessian QP subproblem is in general nonconvex.

Importantly, this indefiniteness is confined to the individual stage blocks: H_k remains block-diagonal across stages and, together with the banded Jacobian induced by the shooting constraints, preserves the characteristic sparse structure of the OCP.

Collecting the stage Hessians H_k and Jacobians and linearizing the constraints of (4.1) at the current SQP iterate yields a structured QP. The subproblem takes the form:

$$\begin{aligned}
& \min_{\substack{\Delta x_0, \dots, \Delta x_N \\ \Delta u_0, \dots, \Delta u_{N-1}}} \sum_{k=0}^{N-1} \left(\frac{1}{2} \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix}^\top \begin{bmatrix} Q_k & S_k^\top \\ S_k & R_k \end{bmatrix} \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix} + \begin{bmatrix} q_k \\ r_k \end{bmatrix}^\top \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix} \right) \\
& \quad + \frac{1}{2} \Delta x_N^\top Q_N \Delta x_N + q_N^\top \Delta x_N \\
& \text{subject to} \quad \Delta x_0 = \Delta \bar{x}_0, \\
& \quad \Delta x_N = \Delta \bar{x}_N, \\
& \quad \Delta x_{k+1} = A_k \Delta x_k + B_k \Delta u_k + c_k, \quad k = 0, \dots, N-1, \\
& \quad \begin{bmatrix} \underline{u}_k \\ \underline{x}_k \\ \underline{d}_k \end{bmatrix} \leq \begin{bmatrix} J_k^{b,u} & 0 \\ 0 & J_k^{b,x} \\ D_k & C_k \end{bmatrix} \begin{bmatrix} \Delta u_k \\ \Delta x_k \end{bmatrix} \leq \begin{bmatrix} \bar{u}_k \\ \bar{x}_k \\ \bar{d}_k \end{bmatrix}, \quad k = 0, \dots, N-1
\end{aligned} \tag{4.3}$$

where, at each stage k , $\Delta x_k \in \mathbb{R}^{n_x}$ and $\Delta u_k \in \mathbb{R}^{n_u}$ denote the state and control deviation variables, and N is the horizon length.

All remaining quantities are obtained by linearizing the OCP at the current SQP iterate (x_k, u_k) with multipliers (π_k, λ_k) :

- Q_k , R_k , and S_k are the state, control, and state–control coupling blocks of the exact Lagrangian Hessian H_k . $q_k = \mathcal{Q}_k(x_k - x_k^{\text{ref}}) + \mathcal{S}_k^\top(u_k - u_k^{\text{ref}})$ and $r_k = \mathcal{R}_k(u_k - u_k^{\text{ref}}) + \mathcal{S}_k(x_k - x_k^{\text{ref}})$ are the corresponding stage-cost gradients, and Q_N , q_N are the terminal Hessian and gradient.
- $A_k = \nabla_{x_k} f_k$ and $B_k = \nabla_{u_k} f_k$ are the Jacobians of the discretized dynamics,

and $c_k = f_k(x_k, u_k) - x_{k+1}$ is the dynamics (shooting) residual at the current iterate; together they define the linearized state-transition equation. At a dynamically feasible iterate $c_k = 0$.

- $\Delta \bar{x}_0 = \bar{x}_0 - x_0$ and $\Delta \bar{x}_N = \bar{x}_N - x_N$ are the residuals of the initial and terminal equality constraints, i.e. the corrections needed to satisfy $x_0 = \bar{x}_0$ and $x_N = \bar{x}_N$ after the step. The terminal equality is present only for the problems that impose it; otherwise this row is absent.
- $J_k^{b,u}$ and $J_k^{b,x}$ are selection matrices picking out the bounded control and state components, with bounds $\underline{u}_k, \bar{u}_k$ and $\underline{x}_k, \bar{x}_k$ (box constraints).
- $C_k = \nabla_{x_k} h_k$ and $D_k = \nabla_{u_k} h_k$ are the Jacobians of the general path constraints, with $\underline{d}_k = \underline{h} - h_k(x_k, u_k)$ and $\bar{d}_k = \bar{h} - h_k(x_k, u_k)$; these constraints couple states and controls. For the affine constraints $\mathcal{C}x_k + \mathcal{D}u_k$ of (4.1), C_k and D_k reduce to the constant matrices $\mathcal{C}$ and $\mathcal{D}$.

4.1.2 Benchmark OCPs

In this subsection, we introduce the formulation of the eight OCPs used to construct the benchmark. Each OCP is obtained by taking a dynamic system provided in [22] and casting it as a regulation or point-to-point transfer problems with problem-specific modifications, which can be described as (4.1). The eight OCPs are named and characterized as follows:

- **Pendulum:** An inverted pendulum on an actuated cart, driven by a horizontal force. It is regulated from the downward to the upright equilibrium subject to an input bound and an additional mixed state–input inequality constraint. Its stage cost further includes a bilinear state–input term, which facilitates the occurrence of indefinite Hessians in the SQP iterations.
- **Acrobot:** Adapted from [23]. A two-link pendulum actuated only at the elbow joint, with joint friction. The state is driven between two prescribed configurations through terminal equality constraints, yielding a strongly nonlinear, underactuated maneuver.
- **Dubins car:** Adapted from [24]. A kinematic unicycle with position and heading as states, actuated by forward velocity and turn rate. It performs a parallel-parking maneuver, a lateral displacement returning to the initial heading, within a narrow corridor imposed by state-box constraints, with a terminal equality constraint on the full pose.
- **Furuta:** Adapted from [25]. A rotary inverted pendulum with experimen-

tally identified parameters and joint damping, actuated by a single base-arm torque. The pendulum is swung up and stabilized at the upright position under tight torque limits and a non-uniform time grid.

- **Flying robot:** Adapted from [26]. A planar free-flying robot with position, orientation and their rates, propelled by four non-negative thrusters that together produce a body-aligned net force and a turning moment. The state is driven toward a reference point—zero position and orientation with a prescribed nonzero translational velocity—under thrust non-negativity and summed-thrust linear constraints.
- **Robot arm:** Adapted from [26]. A three-DOF robot arm (radial extension, rotation, elevation) with configuration-dependent inertia and a cross-axis actuator coupling that is nonlinear in the inputs. A joint angle is regulated to a new set-point under state and input bounds; the input coupling renders the exact Hessian indefinite.
- **Crane:** Adapted from [26]. An overhead crane with hoisting length, trolley position and payload swing angle, actuated by trolley and hoisting accelerations. The load is transferred between two rest positions while respecting a swing-rate bound and a nonlinear region constraint on the payload.
- **Moonlander:** Adapted from [27]. A planar lunar lander with translational and rotational motion under reduced gravity, actuated by two laterally offset thrusters. It performs a soft landing, reaching a ground target with zero terminal velocity, under non-negative, bounded thrust.

Table 4.1: Summary of the evaluated OCP problems. The state dimension n_x , control dimension n_u , and horizon length N span a range of common control problem scales.

Problem	n_x	n_u	N
Pendulum	4	1	25
Acrobot	4	1	15
Dubins car	3	2	25
Furuta	4	1	20
Flying robot	6	4	50
Robot arm	6	3	30
Crane	6	2	40
Moonlander	6	2	30

Besides the different physical systems, These OCPs also cover different state

and control dimensions, as shown in Table 4.1. The state dimension n_x ranges from 3 to 6, the control dimension n_u from 1 to 4, and the horizon length N from 15 to 50. This spread is chosen so that the benchmark spans a range of common problem scales in control problems, which allows the evaluated methods to be assessed across problems of varying size and structure.

4.1.3 Collecting Strategy

Based on the formulated OCPs, we can construct a representative benchmark of indefinite QPs arising from SQP-based NMPC, avoiding the artificial assumptions of randomly generated Hessians and constraints. Specifically, we propose a collection strategy that extracts the QP subproblems directly from intermediate SQP iterations. By logging the linearized dynamics, stage constraints, and indefinite objective approximations exactly as they arise in each iteration, this procedure ensures that every benchmark instance faithfully reflects the numerical conditioning and sparsity encountered in practice.

The collection strategy consists of two steps: *generate* and *collect*, and can be summarized as follows:

1. **Generate:** In this step, we generate iterates by solving the OCP. To obtain iterates at various points during the solution process, we can apply SQP-level regularization techniques (such as PROJECT or MIRROR) or utilize the Gauss-Newton Hessian approximation in `acados`.
2. **Collect:** In this step, we utilize the previously generated iterates as initial guesses for linearization within the solver. By restricting the maximum number of SQP iterations, we can selectively collect indefinite QPs that exhibit an indefinite reduced Hessian.

In the collect step, QPs are all with one failing status code from `acados`, code 4 for infeasible QP, which indicating the indefinite reduced Hessian causing the failing of the Riccati recursion in HPIPM. This is achieved by setting the Riccati algorithm in HPIPM as classical Riccati recursion, i.e. `ric_alg = 0`, instead of the square-root version [19], which requires the reduced Hessian to be positive definite in order to proceed with the Riccati recursion. So when the HPIPM fails to proceed with the Riccati recursion, it indicates that the reduced Hessian is indefinite, and the corresponding QP is collected.

For each problem, we limit the number of near-identical QPs, where similarity is measured by the minimal eigenvalue and the rate of indefinite stages (both

introduced in Section 4.2), so that the number of instances per problem stays within a comparable range and the collected instances remain meaningful. We further use IPOPT [17] as a reference solver and keep only the QPs that it solves successfully. After this filtering, we obtain a total of 175 indefinite QPs from the eight OCPs, the per-problem counts are reported in Table 4.2 and range from 15 to 29.

Table 4.2: Number of indefinite QPs collected from each OCP (175 in total).

Problem	Number
Pendulum	18
Acrobot	15
Dubins car	29
Furuta	19
Flying robot	20
Robot arm	24
Crane	29
Moonlander	21

The QPs are stored in json format and are available at https://github.com/Pandatheon/Primal-dual_IPM.git.

4.2 Benchmark Analysis

In this section, we will introduce some analysis of the collected problem instances for gaining a better understanding of characteristics regarding definiteness in the indefinite OCP QP benchmark. The distribution of indefiniteness is analyzed in Section 4.2.1, which describes the magnitude and coverage of the indefinite reduced Hessians in the collected QPs, and the source of indefiniteness is analyzed in Section 4.2.2, which further illustrates the source of indefiniteness in the collected QPs by analyzing the eigenvalues of the stage Hessians.

4.2.1 Distribution of Indefiniteness

We first characterize the collected QPs along two axes: the *depth* of indefiniteness, measured by the magnitude of the smallest (most negative) eigenvalue $|\lambda_{\min}|$ of the reduced Hessian, and the *width*, measured by the fraction of block reduced Hessians that are indefinite within a QP (the indefinite-stage rate).

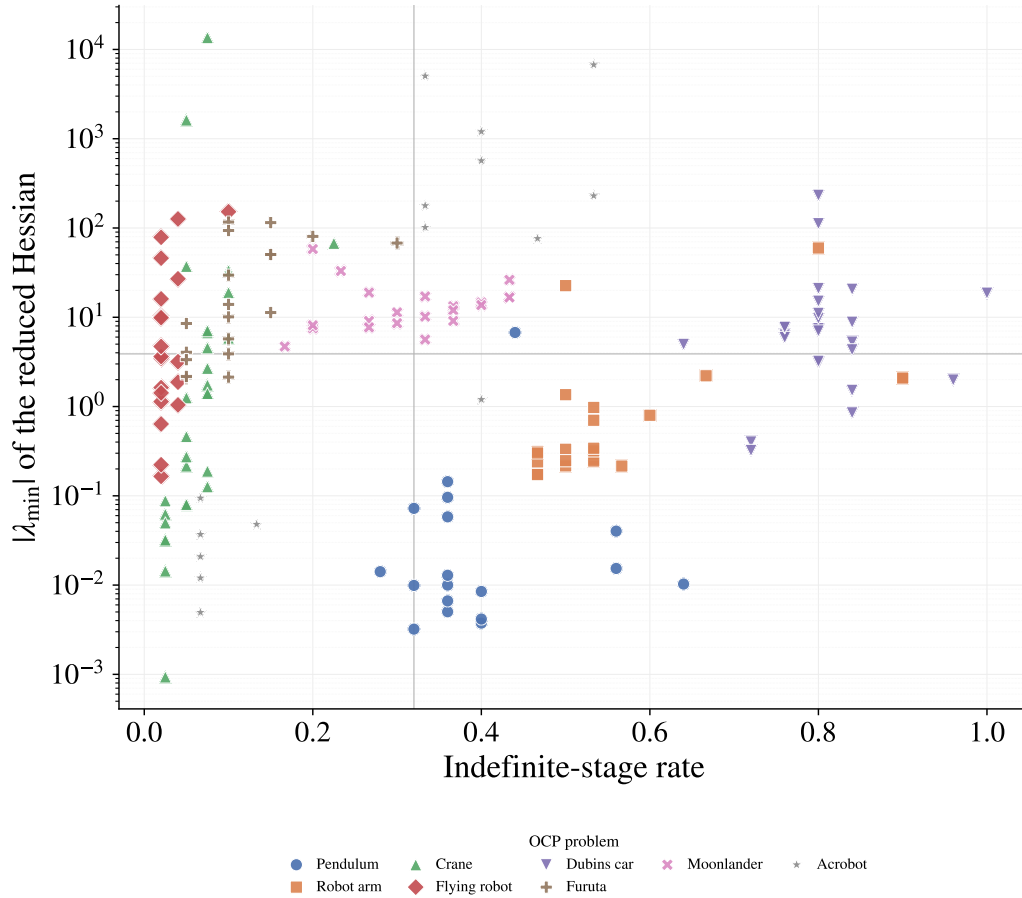


Figure 4.1: Depth ($|\lambda_{\min}|$ of the reduced Hessian, log scale) versus width (indefinite-stage rate) of the collected QPs, coloured by OCP problem.

As seen from Figure 4.1, the collected indefinite QPs do not concentrate in a single regime. They spread over more than seven orders of magnitude in depth ($|\lambda_{\min}|$ ranging from about 10^{-3} to 10^4). And the coverage is strongly problem-dependent: each OCP occupies a characteristic region of the depth–width plane rather than scattering uniformly.

Two complementary major patterns can be observed. Problems that are indefinite in only a small fraction of their stages, such as *Crane*, *Flying robot* and *Furuta*, nevertheless reach very deep negative curvature: their indefinite-stage rates stay below 0.30 (with medians of 0.05, 0.02 and 0.10, respectively), while $|\lambda_{\min}|$ reaches up to 10^2 – 10^4 . Conversely, problems with a high indefinite-stage rate, *Dubins car* and *Robot arm*, with rates ranging from 0.47–1.0, tend to exhibit only moderate depth, with $|\lambda_{\min}|$ clustered around 10^{-1} – 10^1 . Broad and deep indefiniteness thus appear to occur largely separately rather than together.

Some problems also fall in the middle, such as *Moonlander* and *Acrobot*, which occupy intermediate indefinite-stage rates (roughly 0.07–0.53). *Moonlander* keeps its depth in a narrow band ($|\lambda_{\min}|$ between 10^0 and 10^2), whereas *Acrobot* scatters over almost the entire depth range, from near-singular values around 10^{-2} up to nearly 10^4 . They neither confine their negative curvature to a few stages nor spread it across the whole horizon, and thus bridge the two extreme regimes rather than belonging clearly to either.

Taken together, the eight OCPs cover a broad range of the depth–width plane: from narrow-but-deep (*Crane*, *Flying robot*, *Furuta*), through intermediate (*Moonlander*, *Acrobot*), to broad-but-moderate (*Dubins car*, *Robot arm*), with *Pendulum* adding cases of very shallow negative curvature ($|\lambda_{\min}|$ mostly below 10^{-1}) at intermediate indefinite-stage rates. This spread indicates that the collected QP subproblems exercise indefiniteness across qualitatively different regimes rather than a single one.

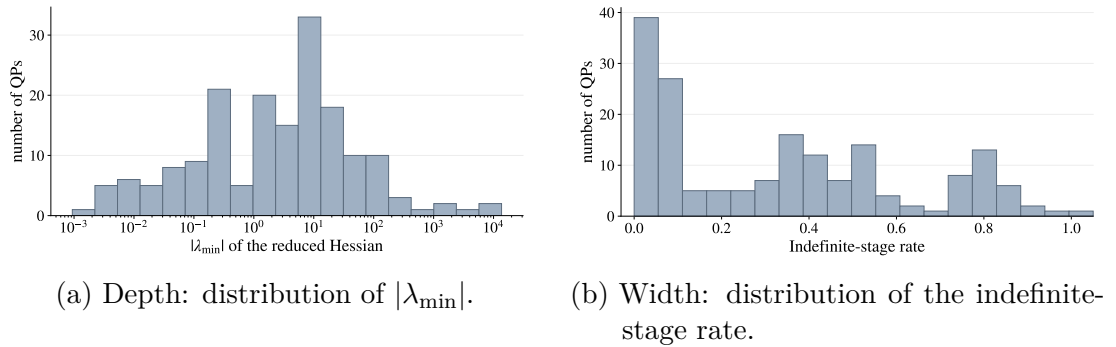


Figure 4.2: Marginal distributions of depth and width over all collected QPs.

Figure 4.2 shows the two marginal distributions of the same data. The depth histogram (Figure 4.2a) is centred around $|\lambda_{\min}| \approx 10$, where most of the indefinite QPs lie, with roughly one third of all instances falling between $10^{0.5}$ and $10^{1.5}$, and decays toward both a near-singular tail near 10^{-3} and a small number of very deep cases at 10^3 – 10^4 , of which only five instances exceed 10^3 . Hence, although extreme cases exist, the negative curvature typically encountered during the SQP iterations is of moderate magnitude.

The width histogram (Figure 4.2b) shows how the indefinite-stage rate is distributed over the collected QPs. The rate covers the full range up to 1, with a large share of QPs being indefinite in only a minority of their stages: close to 40% of the instances have a rate below 0.11. The remainder is spread out with a secondary accumulation around rates of 0.33–0.55 and a further group around

0.72–0.88, while only a single QP is indefinite in essentially all of its stages.

This is consistent with the scatter plot: for most problems only a subset of stages become indefinite, whereas a few problems (*Dubins car*, *Robot arm*) are indefinite across the majority of their stages.

4.2.2 Source of Indefiniteness

Recalling the block reduced Hessian at k stage is calculated as $M_k = R_k + B^\top P_{k+1} B$, which is assembled from two contributions: the control cost block R and the cost-to-go block P .

To understand *where* the indefiniteness originates, we classify every indefinite stage according to which of the two blocks has a negative smallest eigenvalue: *R only* (R indefinite, P definite), *P only* (P indefinite, R definite), and *both* (both indefinite).

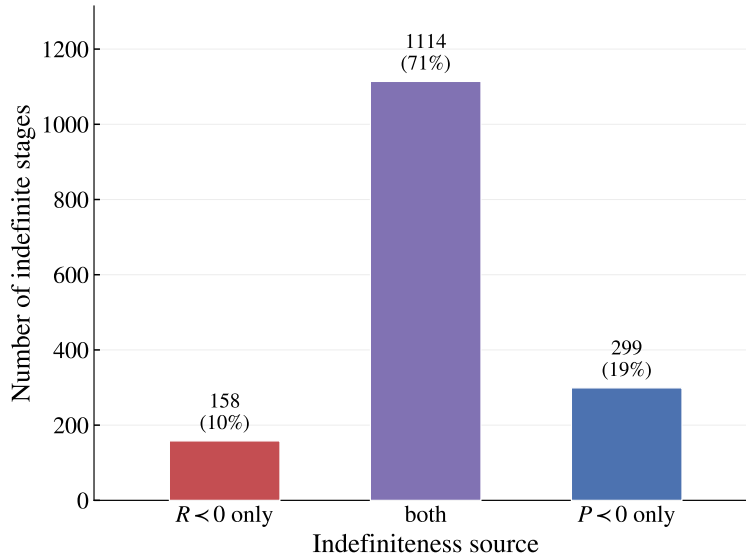


Figure 4.3: Number of indefinite stages by source: control cost R , cost-to-go P , or both.

Figure 4.3 aggregates all indefinite stages across the benchmark. The dominant source is the joint case: 1114 of the 1571 indefinite stages (71%) have *both* R and P indefinite, so in the large majority of cases the negative curvature is not confined to a single block. Among the remaining stages where only one block is indefinite, the cost-to-go block is the more frequent culprit: P alone accounts for 299 stages (19%), nearly twice the 158 stages (10%) in which only the control cost R is indefinite.

Overall, the cost-to-go block P is indefinite more often than the control cost block R , being involved in 1413 of the 1571 indefinite stages (90%) against 1272 (81%) for R , which indicates that the indefiniteness is driven primarily by the cost-to-go term rather than by the control cost.

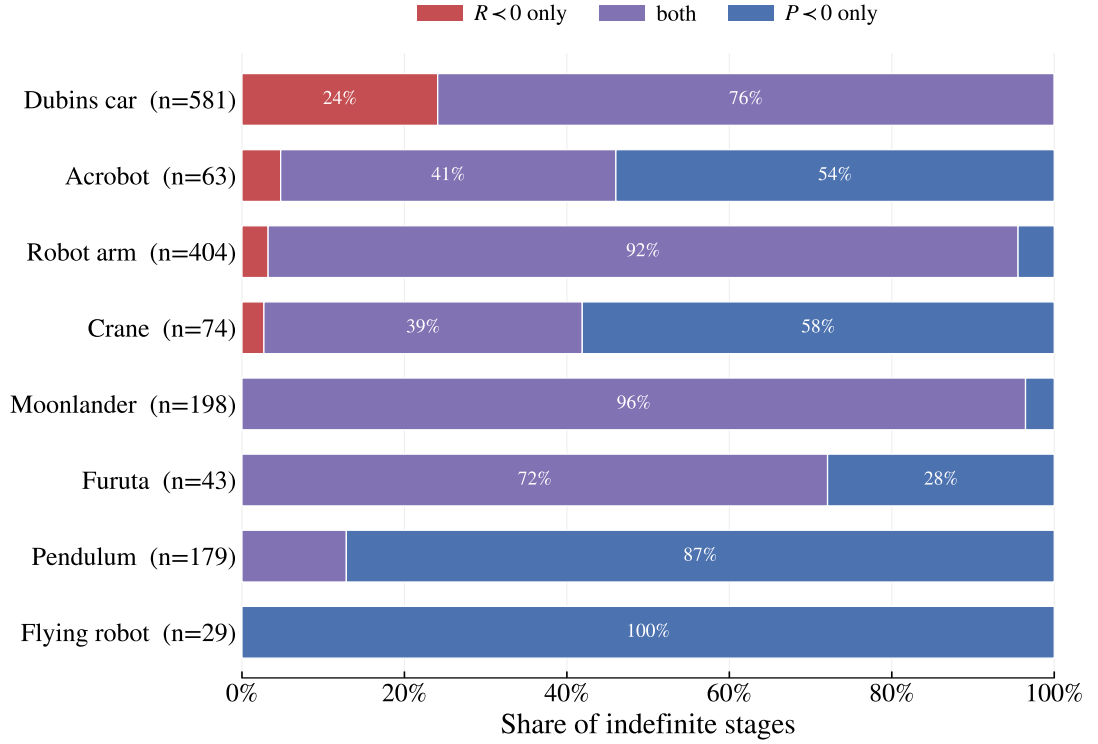


Figure 4.4: Per-problem composition of stage indefiniteness. Each bar shows, for one OCP, the share of stages that are R only, both, P only, (n is the number of collected indefinite stages).

While Figure 4.3 gives the aggregate picture, Figure 4.4 breaks the same classification down per problem and reveals a strongly problem-dependent and clean structure: six of the eight OCPs are dominated by a single category ($\geq 70\%$ of their stages), three of them almost exclusively so ($\geq 90\%$), and only *Acrobot* and *Crane* are genuinely mixed.

As observed from Figure 4.4, most of the problems are governed by the *both* case, including *Moonlander* (96%), *Robot arm* (92%), *Dubins car* (76%) and *Furuta* (72%). Notably, three problems, *Dubins car* ($n = 581$), *Robot arm* ($n = 404$) and *Moonlander* ($n = 198$), contribute by far the most indefinite stages, they largely account for the aggregate dominance of the *both* case reported above.

The P only case dominates the remaining problems: *Flying robot* is indefinite

purely through the cost-to-go block (100%), and *Pendulum* nearly so (87%). The two mixed problems lean the same way: *Crane* splits into 58% *P only* against 39% *both*, and *Acrobot* into 54% *P only* against 41% *both*, so that even where no single category dominates, the cost-to-go block is the more frequent one. Taken over all stages, the cost-to-go block is indefinite in the large majority of cases, whereas the *R only* case is marginal: it appears in only four problems and is negligible for *Robot arm*, *Acrobot* and *Crane* (all below 5%), with *Dubins car* being the sole problem where isolated *R*-block indefiniteness is substantial (24% of its stages, accounting for 140 of the 158 *R only* stages in the whole benchmark). Indefiniteness therefore predominantly involves the cost-to-go block, either alone or jointly with *R*, while *R*-block-only indefiniteness is the exception.

In conclusion, these figures numerically shows that, across the benchmark the cost-to-go block is involved in the vast majority of indefinite stages, a regularization strategy that merely enforces $M \succ 0$ would therefore leave the negative curvature unaddressed in most stages. Since the cost-to-go term enters the reduced Hessian through $M_k = R_k + B^\top P_{k+1} B$, keeping P_{k+1} (and hence M_k) definite is essential for a reliable regularization method, whereas treating the control cost block alone is not sufficient.

Chapter 5

Numerical Experiments

In this chapter, we evaluate the proposed regularization methods numerically and discuss the results. We begin in Section 5.1 with the two metrics used throughout the comparison, the iteration profile and the flop unit estimation, which measure convergence behavior and factorization cost respectively. Section 5.2 then describes the prototype solver on which all experiments are run and establishes that it reproduces the iteration behavior of HPIPm on convex QPs, so that any difference observed later can be attributed to the regularization rather than to the solver itself. Section 5.3 fixes the experiment setup, naming and classifying the regularization methods under comparison and stating the parameters shared by all of them. Section 5.4 presents the numerical comparison, first among the Riccati-based methods themselves and then against full Hessian regularization and inertia correction. Section 5.5 concludes the chapter by discussing what distinguishes the hard instances from the easy ones and why the full Hessian methods perform poorly.

5.1 Experiment Metrics

In this section, we will introduce the metrics used to evaluate the performance of different regularization strategies in our experiments. Since we care about both the convergence behavior and the factorization cost caused by different regularization methods, we will use two complementary metrics: iteration profile and flop unit estimation. The former one, iteration profile, is used to evaluate how many iterations each method takes to converge on a set of test instances, while the latter one, flop unit estimation, is approximated by the number of modified Cholesky factorization of the reduced Hessian, which is used to evaluate the factorization cost of each method.

5.1.1 Iteration Profile

Let $\mathcal{P}$ denote the set of test instances and $\mathcal{M}$ the set of candidate regularization strategies. For a method $m \in \mathcal{M}$ and an instance $p \in \mathcal{P}$, let $i_{m,p}$ be the number of outer iterations at which m satisfies the convergence criterion on p . A run that reaches the iteration cap $i_{\max}$ without converging, or that stalls early, is treated as unsolved and we set $i_{m,p} = \infty$.

The *iteration profile* of method m is

$$\Pi_m(\ell) = |\{p \in \mathcal{P} : i_{m,p} \leq \ell\}|, \quad \ell = 1, \dots, i_{\max}, \quad (5.1)$$

i.e. the number of instances that m solves within a budget of ℓ iterations. Equivalently, $\Pi_m(\ell)/n_{\mathcal{P}}$ is the empirical cumulative distribution of the solve-iteration count over the test set, which plateaus at the solve rate of m rather than at 1. By construction Π_m is non-decreasing in ℓ , and its value at the cap, $\Pi_m(i_{\max})$, equals the total number of instances solved by m under the budget.

The profile is read along two axes. The rate at which $\Pi_m(\ell)$ rises with ℓ reflects the *efficiency* of method m : a curve that climbs steeply at small ℓ solves most instances with few iterations. The height of the plateau, $\Pi_m(i_{\max})$, reflects the *robustness* of the method: it is the number of test instances solved within the prescribed budget. Because every method is evaluated under the same cap $i_{\max}$ and the same convergence criterion, the curves are directly comparable across $\mathcal{M}$.

This metric gives a summary of how reliably and how quickly each strategy converges across the test set.

5.1.2 Flop Unit Estimation

Iteration counts alone do not reflect the work performed per iteration, which differs substantially across regularization strategies: an eigenvalue-based correction and a modified-Cholesky correction may reach convergence in a comparable number of iterations while incurring very different arithmetic cost. To compare methods on a common scale, we express all dense linear-algebra work in a single flop unit.

We take one modified Cholesky factorization of the block reduced Hessian as the unit. For a control dimension n_u , factorizing an $n_u \times n_u$ matrix costs approximately $\frac{1}{3}n_u^3$ flops, hence

$$\text{unit} \approx \frac{1}{3}n_u^3. \quad (5.2)$$

The modified factorization differs from the plain one only by lower-order diagonal updates, so to leading order it also costs one unit.

The remaining primitives are expressed relative to this unit. A symmetric eigenvalue decomposition of the same block costs about $9n_u^3$ flops [28], i.e.

$$\frac{9n_u^3}{\frac{1}{3}n_u^3} = 27 \text{ units.} \quad (5.3)$$

With this unit defined, the arithmetic cost of a solve, including the regularization applied at each iteration, can be expressed on a common scale and accumulated over the iteration profile.

5.2 Prototype Solver Implementation

To unify the comparison, we implemented a prototype solver capable of accommodating all regularization methods under consideration. The Riccati-based primal-dual interior-point solver is implemented in Python, using HPIPM, the state-of-the-art Riccati-based IPM solver for OCP QPs, as a reference. The prototype was designed to reproduce its iteration behavior on convex QPs, where the two solvers should therefore agree. It is available at https://github.com/Pandatheon/Primal-dual_IPM.git.

One implementation detail deserves mention, as it departs from the plain FTB rule stated in Section 2.4. Rather than retracting the maximal step by a fixed fraction η , the primal and dual step sizes α_{prim} and α_{dual} are each obtained from their respective maximal step through the heuristic

$$\alpha = \begin{cases} \alpha_{\max} \left[(1 - \alpha_{\max}) \kappa_{\eta} + \alpha_{\max} \hat{\eta} \right], & \alpha_{\max} < 1, \\ \alpha_{\max}, & \alpha_{\max} = 1, \end{cases} \quad (5.4)$$

where $\alpha_{\max} \in (0, 1]$ denotes the maximal step length for ensuring the positivity of the slack and dual variables, $\kappa_{\eta} = 0.99$ and $\hat{\eta} = 1 - 10^{-7}$. The bracketed factor interpolates linearly between the conservative constant κ_{η} and the near-unit value $\hat{\eta}$, with $\alpha_{\max}$ itself as the interpolation weight. The separate case $\alpha_{\max} = 1$ is needed, otherwise the first branch would scale an unblocked full step by $\hat{\eta}$ and prevent it from ever being taken exactly, thus affecting the local quadratic convergence rate of IPM.

For the comparison to be meaningful, We align the prototype solver with

HPIPM in terms of the algorithmic choices. We therefore define a set of HPIPM settings, referred to in the following as the **CUSTOMIZED** mode, the values are listed in Table 5.1.

Table 5.1: Detailed settings for the defined **CUSTOMIZED** mode.

Parameter	Value	Parameter	Value
mu0	1e1	lam_min	1e-12
alpha_min	1e-12	t_min	1e-12
res_g_max	1e-6	tau_min	1e-12
res_b_max	1e-8	lam0_min	1e-12
res_d_max	1e-8	t0_min	1e-12
res_m_max	1e-6	warm_start	0
dual_gap_max	1e15	abs_form	0
iter_max	30	comp_dual_sol_eq	1
stat_max	30	comp_res_exit	1
pred_corr	1	comp_res_pred	1
cond_pred_corr	1	split_step	1
itref_pred_max	0	m_safe	0.5
itref_corr_max	0	var_init_scheme	0
reg_prim	0.0	t_lam_min	2
square_root_alg	0	t0_init	2
lq_fact	0	update_fact_exit	1

As one can notice, the **CUSTOMIZED** mode largely coincides with the **SPEED** mode of the original HPIPM, differing in two respects. First, the square-root algorithm is disabled (`square_root_alg=0`), so that the Riccati recursion is carried out directly rather than on Cholesky factors. Second, the primal and dual step sizes are computed separately (`split_step=1`) instead of sharing a single step length. In both modes no iterative refinement is applied to the predictor or corrector steps (`itref_pred_max=itref_corr_max=0`) and no default primal regularization is added (`reg_prim=0`), while Mehrotra’s predictor–corrector is retained and applied conditionally (`pred_corr=cond_pred_corr=1`).

To verify that the two implementations do agree, we collected 40 convex QP subproblems, five from each of the 8 OCPs in the benchmark, and compared the iteration behavior of the prototype solver against that of **CUSTOMIZED**-mode HPIPM. The resulting iteration profiles are shown in Figure 5.1.

Both solvers solve all 40 instances in the exact same number of iterations for each instance, resulting in performance profiles that coincide across the whole budget range, from 4 up to 22 iterations. On convex QPs the prototype solver

therefore reproduces the iteration behavior of HPIPM, which verifies the prototype as a reliable baseline and allows us attribute the differences observed in the experiments to the regularization rather than to the solver.

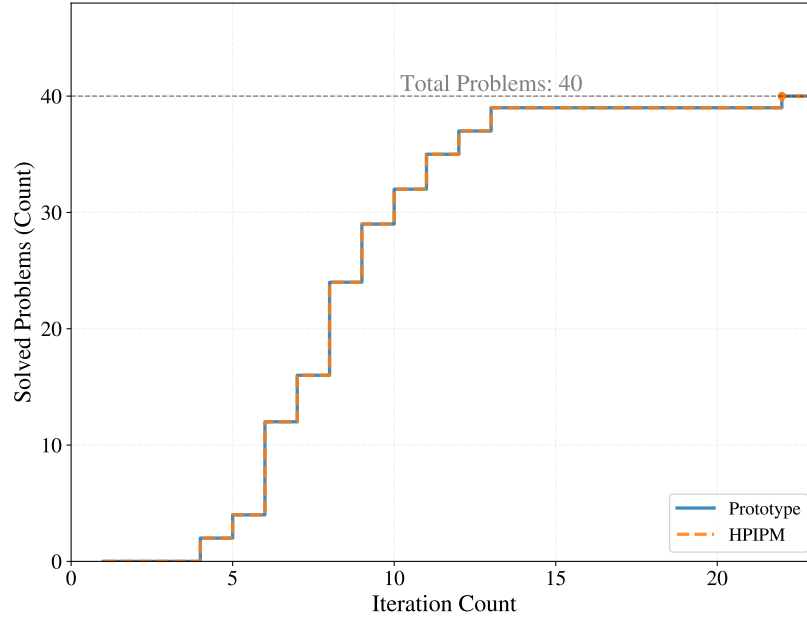


Figure 5.1: Iteration profiles on 40 convex QPs. The prototype solver is compared with HPIPM, showing identical iteration behavior.

5.3 Experiment Setup

In this section, the settings preparation for the experiment are introduced. First, based on the previously discussed methods, we will name and classify the regularization methods considered in the experiment, then we will introduce the general setup of the experiments, including the convergence criterion and step-size control strategy.

5.3.1 Experimented Regularizations

Before turning to the results, we fix the set of regularization methods entering the comparison, together with the naming convention used throughout this chapter.

Following the classification in Section 3.1, the methods are grouped by their *target*, that is, by the matrix whose indefiniteness the correction is derived from and to which the correction is directly applied, rather than by how the correction itself is computed. This yields three families: full Hessian regularization,

Riccati-based regularization, and inertia correction. The numerical strategies of Section 3.1.1, namely spectral projection (**PROJECT**), eigenvalue mirroring (**MIRROR**) and the Gill–Murray–Wright modified Cholesky factorization (**GMW**), are largely independent of the matrix they act on and can therefore be paired with each target.

Every method is accordingly named in the form **TARGET_STRATEGY**, so that both factors can be read off directly from the label, and the comparison can be carried out either across targets at a fixed strategy or across strategies at a fixed target.

Full Hessian Regularization The first family follows Section 3.1.2 and enforces positive definiteness of the full Hessian $\tilde{H}$ in the KKT system. The regularization is also triggered by the failure of the Cholesky factorization in Riccati recursion, the regularized full Hessian is obtained by stage-wisely regularizing each block $\tilde{H}_k$ and the Riccati recursion is re-attempted with it. The three variants differ only in how the block adjustment E_k is formed:

- **H_PROJECT**: the negative and near-zero eigenvalues of each $\tilde{H}_k$ are clipped to ϵ , which gives the smallest perturbation in Frobenius norm that restores positive definiteness.
- **H_MIRROR**: the eigenvalues of each $\tilde{H}_k$ are replaced by their absolute values (floored at ϵ), which preserves the magnitude of the curvature along every eigendirection at the price of a larger modification.
- **H_GMW**: the diagonal pivots of each $\tilde{H}_k$ are bounded from below during the $\text{LDL}^\top$ factorization, which avoids an eigendecomposition altogether and keeps the entries of the triangular factor bounded, but yields a non-minimal, ordering-dependent perturbation.

Riccati-based Regularization The second family is the one proposed in Section 3.2 and acts on the reduced Hessian $\tilde{M}$. The algorithms are given in Algorithms 4 and 5, where the regularization is applied stage-wise to the block reduced Hessian $\tilde{M}_k$ during the backward pass, when the failure of the Cholesky factorization of $\tilde{M}_k$ occurs.

The two algorithms differ in which matrices of the backward pass the adjustment is derived from and applied to, giving two targets within this family:

- **M**: only the block reduced Hessian $\tilde{M}_k$ is corrected, as in Algorithm 4. The correction is purely local and serves as a minimal safeguard for the

factorization.

- **PM**: the cost-to-go matrix P_{k+1} entering the stage is corrected as well, as in Algorithm 5 introduced in this work, so that the propagation of negative curvature analysed in Section ?? is accounted for consistently in $\tilde{M}_k$, Γ_k and P_k .

Pairing each target with the three strategies gives the six methods compared in this chapter. Each method accordingly answers a specific question in the comparison:

- **M_PROJECT**: the reference point of the family. A minimum-norm correction applied locally is the naive and cheapest way to enforce positive definiteness, and is the configuration that the analysis of Section ?? predicts to be the most exposed to condition-number growth.
- **M_MIRROR**: whether abandoning the minimum-perturbation rule is by itself sufficient, with the propagation left untreated.
- **M_GMW**: the same question as **M_MIRROR**, but with the correction on the inertia of the factorization rather than on the spectrum, which indicates whether the cheaper Type-II correction can stand in for mirroring.
- **PM_PROJECT**: conversely, whether regularizing the cost-to-go matrix helps even when the minimum-perturbation rule is retained, so that this case can be discussed rather than merely argued for.
- **PM_MIRROR**: both insights combined, and thus the method the analysis of Section ?? recommends.
- **PM_GMW**: combines both insights with a cheaper factorization-based correction to explore the trade-off between robustness and decomposition cost compared to **PM_MIRROR**.

Inertia Correction The third family is the Levenberg–Marquardt style inertia correction used in IPOPT [17], which we denote by **INERTIA_LM** and take as the practical reference point for the comparison.

In this strategy, a scalar multiple of the identity $\delta_w I$ is added stage-wise to the full Hessian, i.e. each full Hessian block $\tilde{H}_k$ is replaced by $\tilde{H}_k + \delta_w I$, and the Riccati recursion is repeated with increasing δ_w until it succeeds. Only the primal regularization from IPOPT is retained, as dual regularization is not the primary focus of this work.

Notably, the method sits between the two preceding families, as it has no distinct target matrix in the above sense. Being a uniform multiple of the identity,

the shift is not tied to any particular direction, so the perturbation remains a uniform shift along every direction for both the full and the reduced Hessian. Accordingly, as $\delta_w I$ is applied uniformly to Q_k and R_k in full Hessian blocks $\tilde{H}_k$, it enters the Riccati recursion as the same additive term on the reduced Hessian blocks $\tilde{M}_k$ and P_k .

The trial sequence follows IPOPT as well: the unmodified matrix is attempted first ($\delta_{\text{LM}} = 0$), the first correction of a solve starts from δ_w^0 and is then scaled by the accelerated factor $\bar{\kappa}_w^+$, subsequent corrections are scaled by κ_w^+ until the Riccati recursion succeeds or the threshold δ_w^{max} is exceeded, and a successful value is stored and reduced by κ_w^- as the starting point of the next iteration. The parameters are set to the IPOPT defaults and are listed in Table 5.2.

Table 5.2: Inertia Correction (IC) Parameters

Symbol	Value	Description
δ_w^0	10^{-4}	Initial Hessian perturbation parameter
δ_w^{min}	10^{-16}	Minimum allowable regularization parameter
δ_w^{max}	10^{16}	Maximum allowable regularization threshold
κ_w^-	$1/3$	Factor to decrease perturbation upon success
κ_w^+	8	Factor to increase perturbation after inertia failure
$\bar{\kappa}_w^+$	10^2	Accelerated scaling factor for severe indefiniteness
δ_{LM}	0.0	Initial LM shift (IC-1: attempt unmodified matrix first)

Taken together, the three families span ten methods, which share the same solver, the same benchmark and the same parameters introduced in the next subsection,

5.3.2 General setup

All ten methods are run under the same conditions. The barrier parameter is updated by Mehrotra probing (`pred_corr`= 0) and the step size is controlled by the FTB rule followed by the two-phase line search, while the remaining settings are collected in Table 5.3.

A run terminates either when all four residuals fall below their tolerances or when the iteration budget i_{max} is exhausted, in which case the instance counts as unsolved.

Table 5.3: Settings shared by all experiments.

Symbol	Value	Role
r_{stat}	10^{-6}	Tolerance on the stationary residual
r_{eq}	10^{-8}	Tolerance on the equality residual
r_{ineq}	10^{-8}	Tolerance on the inequality residual
r_{comp}	10^{-6}	Tolerance on the complementarity residual
i_{max}	120	Iteration budget, beyond which the instance counts as unsolved
$n_{\mathcal{P}}$	175	Nonconvex QP instances in the test set
ϵ	10^{-4}	Floor on the corrected eigenvalues and pivots, shared by all strategies
β	0.5	Backtracking factor, $\alpha_{\text{prim}} \leftarrow \beta \alpha_{\text{prim}}$
n_{max}	10	Maximum backtracking iterations
κ_s	10^{-1}	Switching constant, $\alpha_{\text{prim}}(-\rho) > \kappa_s \theta^2$
γ	10^{-4}	Armijo constant for the f-phase, $f^+ \leq f + \gamma \alpha_{\text{prim}} \rho$

5.4 Numerical Comparison of Primal Regularizations

In this section, we report the numerical results of the ten regularization methods on the nonconvex QPs of the benchmark. The comparison proceeds in three stages, each narrowing the field. We first compare the six Riccati-based methods against one another, in order to identify the strongest representatives of the family. These are then set against the full Hessian regularizations, and finally against inertia correction, which serves as the practical reference point.

5.4.1 Comparison among Riccati-based regularizations

We first compare the proposed Riccati-based regularization methods, which are the main focus of this work. As a baseline we also include the unregularized recursion (NO_REGULARIZE).

The iteration profile in Figure 5.2 reports, for each method, the number of instances solved within a given iteration budget, where two trends stand out. First, each **PM** dominates its counterpart **M** across the whole budget range, second, among the regularization strategies, **MIRROR** and **GMW** are clearly more

effective than **PROJECT**. These trends validate the insights mentioned in Section ?? . Among all the methods, **PM_MIRROR** tops the profile, solving 171 of the 175 instances, with **PM_GMW** close behind at 168.

Besides, it is interesting to note that the unregularized recursion can also solve a non-negligible fraction of the test set (31 of the 175 instances), which can be explained by the curvature contribution of the constraint that help to make the reduced Hessian positive definite in some instances.

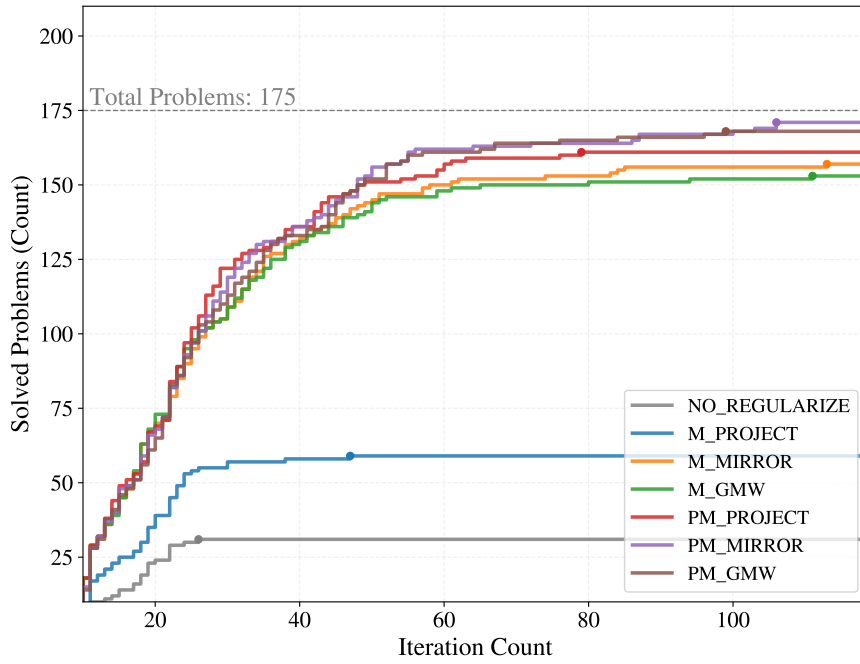


Figure 5.2: Iteration profile of the six Riccati-based methods over the 175-instance test set: number of instances solved within a given iteration budget.

Figure 5.3 complements this picture by showing the distribution of the worst-case decomposition cost in flop estimation, annotated with the maximum observed cost and the number of failures for each method. Since only the successful instances and maximal iteration instances are counted, the **M_PROJECT** will be excluded from the analysis, due to the frequent failing on 116 of the 175 instances.

In general, the **PM** incur a higher and more dispersed cost, reflecting the extra factorization work introduced by the cost-to-go term, yet they are markedly more robust and thus the focus of the comparison. In Figure 5.3, **PM_MIRROR** is the most robust method with only 4 unsolved problems, and **PM_GMW** follows with 7 while keeping its worst-case cost more than an order of magnitude below **PM_MIRROR** (5.7×10^4 versus 9.4×10^5 flops) and more than a factor of 40 below

PM_PROJECT (2.4×10^6). Notably, though the PM_PROJECT and PM_MIRROR belong to the same class of regularization strategy, the worst-case cost of PM_PROJECT is higher than that of PM_MIRROR, which can be explained by the issue of inadequate regularization in the minimum perturbation strategy, so more frequency is needed in the PM_PROJECT case.

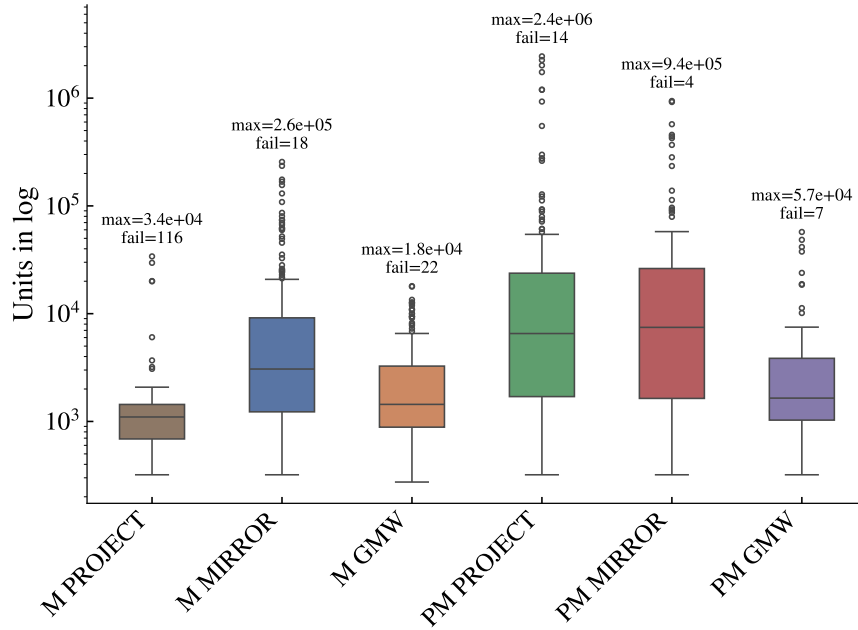


Figure 5.3: Worst-case decomposition cost of the six Riccati-based methods (flops, log scale). Annotations report the maximum observed cost and the number of unsolved instances.

Taken together, two figures first reveal a robustness ordering, $\text{PM_MIRROR} > \text{PM_GMW} > \text{PM_PROJECT} > \{\text{M_MIRROR}, \text{M_GMW}\} > \text{M_PROJECT}$, in which every PM outranks every plain M. If reliability is the priority, PM_MIRROR is the safest choice, as it leaves the fewest instances unsolved. If a balance between reliability and computational cost is preferred, PM_GMW is the more attractive option: it recovers almost all of the robustness of PM_MIRROR (7 unsolved instances against 4) at a worst-case cost roughly $16\times$ smaller. We will therefore focus on PM_MIRROR and PM_GMW in the following comparisons with other regularization families.

5.4.2 Comparison with Full Hessian Regularizations

Having identified `PM_MIRROR` and `PM_GMW` as the strongest Riccati-based methods, we now compare them against the full Hessian regularizations.

The iteration profile in Figure 5.4 shows the two Riccati-based methods sitting well above every full-Hessian variant across the whole iteration budget. `PM_MIRROR` and `PM_GMW` leave only 4 and 7 instances unsolved, whereas each H method leaves a substantial number of instances unsolved even at the largest budget. This suggests that regularizing the full Hessian does not recover the reliability or efficiency obtained by regularizing within the Riccati recursion.

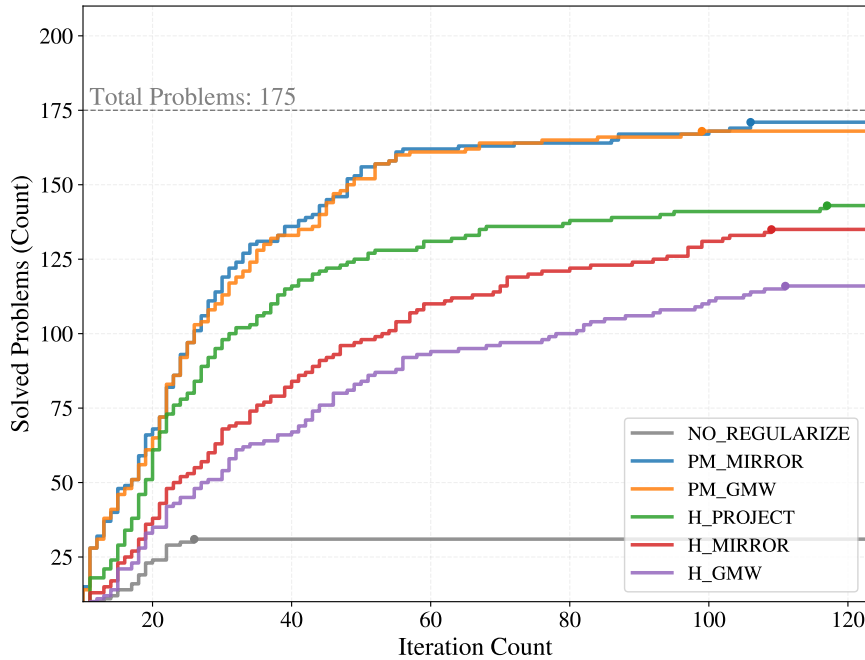


Figure 5.4: Iteration profile of the two strongest Riccati-based methods (`PM_MIRROR`, `PM_GMW`) against the full Hessian regularizations (`H_PROJECT`, `H_MIRROR`, `H_GMW`).

In Figure 5.5, the gap between Riccati-based methods and Full Hessian methods is even more pronounced, and it runs in the Riccati-based methods' favour on *both* axes. In robustness, `PM_MIRROR` fails on only 4 instances and `PM_GMW` on 7, while the full-Hessian methods fail on 32 (`H_PROJECT`), 40 (`H_MIRROR`), and 59 (`H_GMW`) instances. In cost, the full-Hessian variants are one to several orders of magnitude more expensive: their worst-case costs reach 1.5×10^7 flops for `H_PROJECT` and 2×10^7 for `H_MIRROR`, and even the cheapest full-Hessian option, `H_GMW`, has a median cost roughly two orders above that of

PM_GMW, yet it is also the least robust of the methods compared here. The full Hessian methods thus are considered neither speed nor reliability relative to the Riccati-based approach.

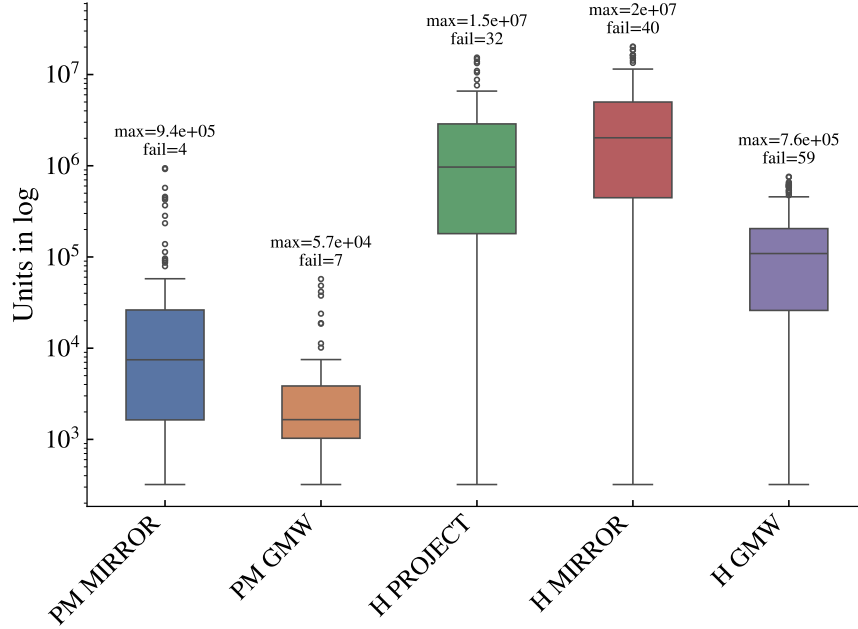


Figure 5.5: Worst-case decomposition cost of the two strongest Riccati-based methods against the full Hessian regularizations (flops, log scale). Annotations as in Figure 5.3.

Unlike the within-family comparison, where robustness and cost traded off against one another, the Riccati-based methods are simultaneously more robust and cheaper than every full-Hessian counterpart in Figure 5.5. This supports the central premise of the approach, namely that regularizing the reduced Hessian by exploiting the stagewise structure of the problem is preferable to regularizing the full Hessian. Consequently, PM_MIRROR and PM_GMW remain the recommended methods, PM_MIRROR when reliability is paramount, and PM_GMW when a lower and more predictable decomposition cost is desired at the price of a handful of unsolved instances.

5.4.3 Comparison with Inertia Correction

Finally, we compare the recommended Riccati-based methods against inertia correction (INERTIA_LM), which is widely used in practice and serves as a reference point for the comparison.

The iteration profiles in Figure 5.6 are close for the three regularized methods, with PM_MIRROR and PM_GMW solving the most instances (171 and 168) and INERTIA_LM tracking them closely across the budget range (160). However, the PM_MIRROR and PM_GMW generally sits slightly above INERTIA_LM, indicating less iteration is needed to solve the same number of instances.

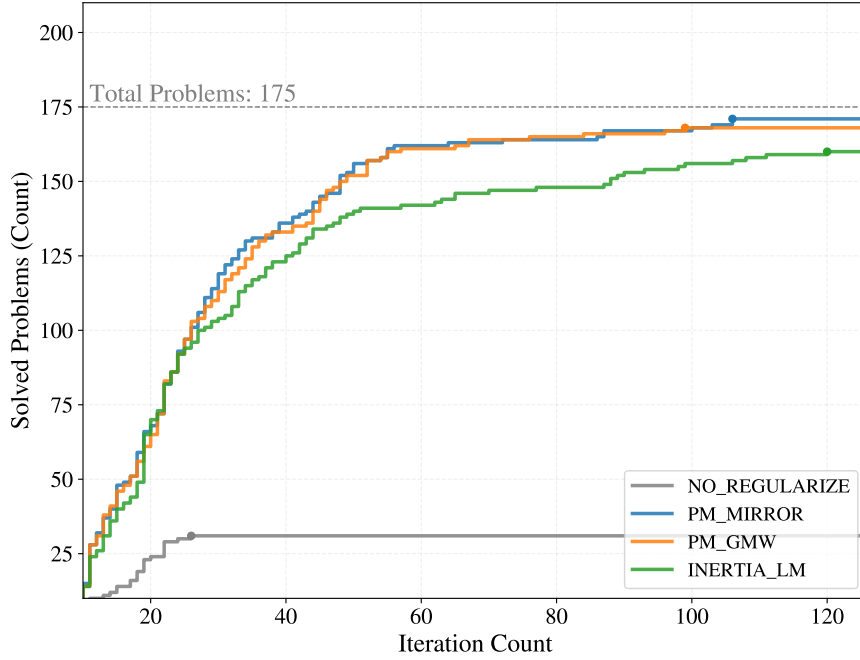


Figure 5.6: Iteration profile of PM_MIRROR and PM_GMW against inertia correction (INERTIA_LM).

The decomposition-cost distribution in Figure 5.7 draws the distinction more finely. Inertia correction is the cheapest of the three in the worst case, with a maximum cost of 2.4×10^4 flops, below both PM_GMW (5.7×10^4) and PM_MIRROR (9.4×10^5). In terms of median cost, however, INERTIA_LM and PM_GMW are essentially on par, with PM_GMW marginally lower. Their robustness differs more clearly, however: INERTIA_LM fails on 15 instances, against 7 for PM_GMW and 4 for PM_MIRROR.

The comparison thus no longer reduces to an even trade-off between INERTIA_LM and PM_GMW. INERTIA_LM retains a single advantage, the lowest worst-case cost, but a narrow one: a factor of roughly 2.4 below PM_GMW, against which PM_GMW is in turn slightly cheaper at every quartile of the distribution and solves eight more instances.

Inertia correction is therefore preferable only where a hard bound on the

per-decomposition cost outweighs the number of solved instances. **PM_MIRROR**, meanwhile, continues to stand apart as the most robust method of the three, and strictly so: the four instances it leaves unsolved are also unsolved by **PM_GMW**, so its additional robustness costs nothing in reliability elsewhere, albeit at a substantially higher worst-case cost.

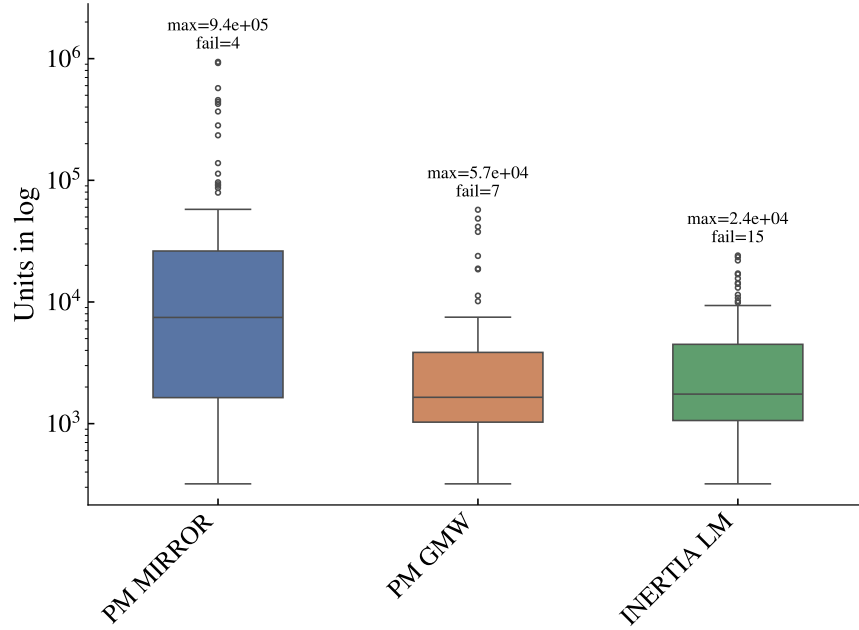


Figure 5.7: Worst-case decomposition cost of **PM_MIRROR** and **PM_GMW** against inertia correction (flops, log scale). Annotations as in Figure 5.3.

5.5 Discussion

The comparison in Section 5.4 ranked the methods by the number of instances they leave unsolved. That number alone, however, says nothing about *how* a method fails or *where* it fails, and both turn out to be more informative than the count itself. This section therefore resolves the same failures along two further axes. Section 5.5.1 splits them by the status with which the solve terminated, which separates a numerical breakdown of the regularized recursion from a mere exhaustion of the iteration budget. Section 5.5.2 splits them by the OCP each instance was collected from, which identifies what distinguishes the hard instances from the easy ones and explains why the full Hessian family performs as poorly as it does.

5.5.1 Failure by status

A run that does not converge terminates in one of three ways, which we record as its exit status and Figure 5.8 breaks down the failures of each method accordingly:

- *Unregularized*: the Cholesky factorization fails *again* after the regularization has been applied, usually caused by numerical indefiniteness.
- *Max iterations reached*: the residuals did not fall below their tolerances within the budget $i_{\max} = 120$.
- *NaN encountered*: an infinite or undefined value appeared in the residuals, so that the iterate itself has overflowed.

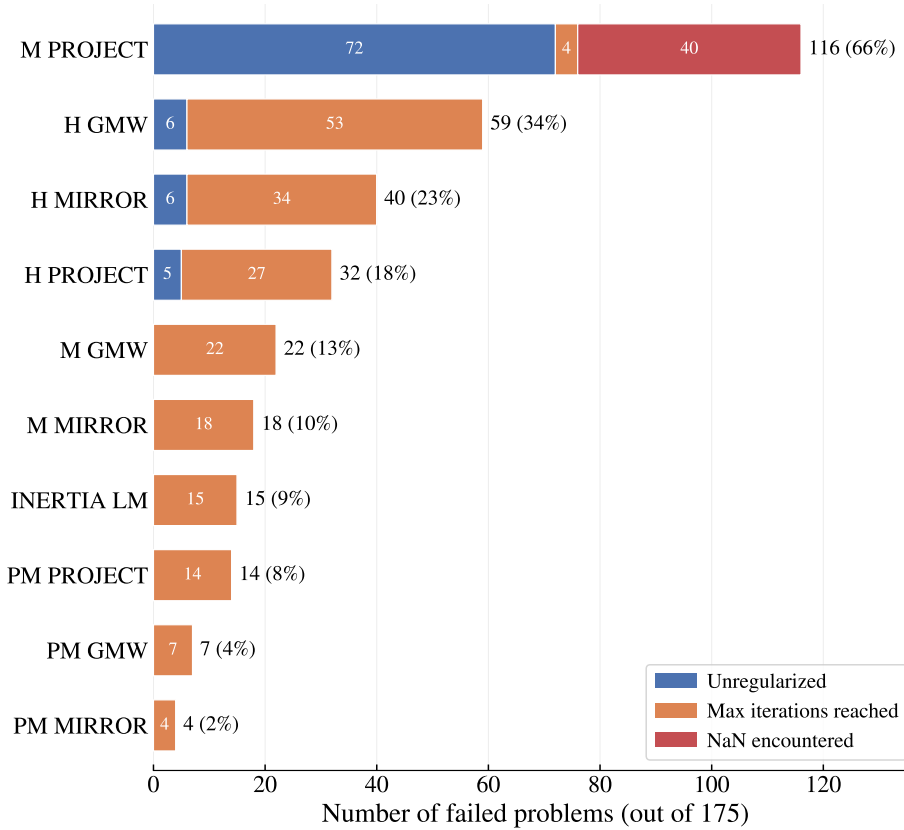


Figure 5.8: Failures of each method by exit status over the 175-instance test set. The bars are stacked by status and annotated with the total number of unsolved instances and the corresponding share of the test set.

In Figure 5.8, the **M_PROJECT** is the only method experiencing numerical breakdowns, with 116 of its 175 instances unsolved. 112 of its 116 failures stem from numerical issue, while only 4 exhaust the iteration budget. This confirms the analysis in Section ??, where ill-conditioning propagates into numerical in-

definiteness or overflow. Adopting either remedy identified there resolves these breakdowns: abandoning the minimum-perturbation rule (**M_MIRROR**, **M_GMW**) or adding the cost-to-go correction (**PM_PROJECT**) eliminates all NaN and unregularized failures, leaving only iteration-limit terminations (18, 22, and 14 failures, respectively). Combining both remedies, as done in **PM_MIRROR** and **PM_GMW**, further reduces the remaining iteration-limit failures.

The full Hessian family also breaks down, failing on 5, 6, and 6 instances across variants, but these counts remain largely uniform, with the family instead distinguished by max iteration reached (27, 34, and 53). Neither the backward induction nor the block regularization itself is at fault, but due to the cost-to-go matrix P_k , which is not modified by any **H** strategy.

Near convergence in IPM, the barrier term $\Lambda_k T_k^{-1}$ grows extremely large as the slack variables approach zero. Through the backward Riccati recursion, this directly drives $\|P_k\|_2$ to become very large. Although the roundoff error generated in the Schur complement update remains of normal relative size, the 2-norm condition number $\kappa_2(P_k)$ inflates so large that $\kappa_2(P_k)\epsilon_{\text{mach}} \geq 1$, where ϵ_{mach} denotes the machine precision. Consequently, such a matrix cannot be represented as positive definite in double precision, and P_k is computed as indefinite regardless of how it is assembled.

The Riccati-based methods experience the same indefiniteness in P_k , yet their execution does not rely on $P_k \succ 0$: enforcing $\tilde{M}_k \succ 0$ immediately prior to Cholesky factorization ensures a factorizable system. The solver exit status therefore records whether indefiniteness is explicitly detected rather than whether it occurs in intermediate quantities. In this context the **PM** formulations outperform the plain **M** variants, which leave P_{k+1} untouched, because stage-wise regularization of P_{k+1} suppresses the backward accumulation of numerical indefiniteness across the horizon.

Accordingly, for the six methods that never suffer factorization breakdowns, namely the Riccati-based methods other than **M_PROJECT** together with **INERTIA_LM**, all failures reduce strictly to exhausted iteration limits, which plausibly reflect an inadequate regularization scale or an ineffective search direction.

5.5.2 Failure by category

Figure 5.9 resolves the same failures by the OCP each instance was collected from. The cell entries are the numbers of unsolved instances and the shading is the failure

rate within the column, so that categories of different size remain comparable, and the eight columns are ordered by size and sum to the 175 instances of the test set.

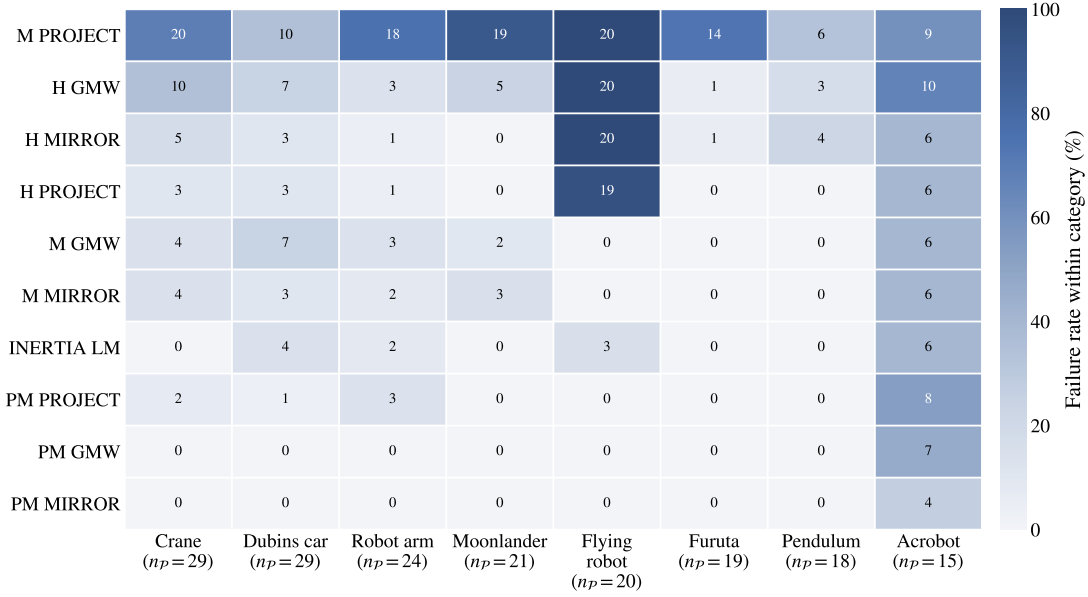


Figure 5.9: Failures of each method by OCP category. The cell entries give the number of unsolved instances and the shading gives the failure rate within the column, where n_p is the number of instances collected from that OCP. The rows are ordered by total failure count and the columns by size.

The first thing the figure shows is that failures of the stronger methods concentrate in the *Acrobot* column. It accounts for all failures of **PM_MIRROR** (4/15) and **PM_GMW** (7/15), and the majority for **PM_PROJECT** (8/14) and **INERTIA_LM** (6/15). Across all ten methods, *Acrobot* failures vary narrowly between 4 and 10 despite total failures ranging from 4 to 116, indicating that these failures stem from instance difficulty rather than regularization. Notably, the four instances unsolved by **PM_MIRROR** are unsolved by all ten methods, hence, **PM_MIRROR** solves the union of all solved instances across the benchmark.

Further, by combining the Figure 4.1, we could speculate what makes that the instance hard is neither depth nor width on its own, but their conjunction. Among the six instances where the reduced Hessian has very negative eigenvalues, the two *Crane* instances have low indefinite rates (0.05 and 0.07) despite reaching $|\lambda_{\min}| \approx 1.4 \times 10^4$, every method except **M_PROJECT** and **H_GMW** solves both, indicating the case that a large correction is needed, a deep but isolated indefinite

stage is corrected once and the recursion recovers. In contrast, the four unsolved *Acrobot* instances sustain $|\lambda_{\min}| > 250$ across 0.33 to 0.53 of the horizon. Applying large correction repeatedly over many stages on the Hessian perturbs the Newton system, preventing the solver from converging within the iteration budget. While Figure 4.1 shows that most instances are either narrow-and-deep or broad-and-moderate, *Acrobot* uniquely combines both attributes, defining the empirical limit of solvability on this benchmark.

Interestingly, in the heatmap, *Flying robot* appears as a single dark column for the full Hessian family, where H_PROJECT, H_MIRROR and H_GMW fail on 19, 20 and 20 of the 20 instances, while every Riccati-based method except M_PROJECT solves all of them. We can further refer to the Figure 4.4 for explanation. In *Flying robot*, indefiniteness is confined to the cost-to-go matrix P (median $|\lambda_{\min}(P)| \approx 1.5 \times 10^3$) and affects only $\sim 2\%$ of stages, while $R_k \succ 0$ holds throughout. Because full Hessian methods cannot access the intermediate recursion matrix P , they enforce positive definiteness indirectly by perturbing $\tilde{Q}_k$ across the entire horizon. This unnecessary alteration of the unaffected 98% of stages severely perturbs the Newton system, causing iteration-budget exhaustion. This is the sharpest single piece of evidence for the premise of Section 3.2, namely that the indefiniteness should be treated where it surfaces in the recursion rather than on the full Hessian.

INERTIA_LM exhibits characteristics intermediate between full Hessian and Riccati-based methods. It adds a uniform shift $\delta_w I$ to both $\tilde{Q}_k$ and $\tilde{R}_k$, which is algebraically equal to adding directly into P in the Riccati recursion. It thereby succeeds to solve 17 of the 20 *Flying robot* instances. By providing a non-minimal perturbation and adjusting both P and $\tilde{M}$ simultaneously, its overall solve rate aligns with the PM family rather than the H family. Its primary limitation, however, is the absence of stage-wise localization: the scalar shift δ_w must match the magnitude of the global minimum negative eigenvalue and is applied across all time steps. This global over-regularization accounts for its remaining failures on instances with high indefinite-stage rates (4 in *Dubins car*, 2 in *Robot arm*, and 6 in *Acrobot*).

The heatmap also refines the M versus PM comparison of Section 5.4. On *Flying robot* the plain M methods are already perfect, because R_k is positive definite there and the indefinite-stage rate is so low that a single local correction is applied and the recursion moves on. Their failures appear instead on *Dubins car* (7 for M_GMW), *Crane*, *Robot arm* and *Moonlander*. Three of these four are

the OCPs dominated by the *both* case, where R_k and P_{k+1} are indefinite at the same stage and the correction is triggered repeatedly. **PM_GMW** clears all four of these columns, and **PM_MIRROR** clears them as well. The benefit of regularizing the cost-to-go matrix is therefore driven by width rather than by depth: it matters precisely when the correction is applied often enough for the contamination of P_k to accumulate across stages, which is the propagation predicted in Section ??.

Together, these results address both central questions of this chapter. First, intractable instances require the conjunction of large negative eigenvalues and high temporal width, a regime confined here to *Acrobot*. Second, full Hessian methods underperform because enforcing the condition $\tilde{H}_k \succ 0$ is overly conservative, imposing it globally alters state curvature even when the indefiniteness is confined to small rate of stages, as in *Flying robot*. Conversely, regularizing directly within the Riccati recursion resolves both limitations by modifying only the indefinite matrices at the specific stages where non-positive eigenvalues arise.

Chapter 6

Conclusion and Outlook

This thesis regularized the indefinite QPs that arise in exact-Hessian SQP for NMPC at the QP level, inside the interior point solver, where the barrier curvature is already taken into account and only the reduced Hessian needs to be positive definite. We showed that the Riccati backward pass is a block $\text{LDL}^\top$ factorization of the reduced Hessian, so that indefiniteness is detected and corrected stage-wise wherever the Cholesky factorization of $\tilde{M}_k$ fails, while the linear complexity of the recursion in the horizon length is preserved. We further showed that the minimum-perturbation rule propagates negative curvature through the cost-to-go matrix to the preceding stages. This led to an improved strategy that uses a non-minimal correction and also regularizes the cost-to-go matrix, and we proved that the regularization becomes inactive close to the solution, so that the local convergence rate of the interior point method is retained.

On a benchmark of 175 indefinite QPs from eight OCPs, whose analysis showed that the cost-to-go matrix is involved in most indefinite stages, **PM_MIRROR** left 4 instances unsolved and **PM_GMW** 7 at a lower cost, against 32 to 59 for full Hessian regularization and 15 for inertia correction. The full Hessian methods failed on almost every *Flying robot* instance, where the indefiniteness is confined to the cost-to-go matrix at a few stages, and the instances left unsolved by all methods combine deep negative curvature with a large share of indefinite stages.

Several questions remain open:

- **Regularization magnitude.** The eigenvalue floor ϵ is fixed, and the remaining failures occur where large corrections are applied at many stages. Adapting the magnitude, e.g. to the conditioning of the corrected blocks or to the barrier parameter, could improve robustness.
- **Other MPC formulations.** In time-optimal and economic MPC, the cost

provides little positive curvature, so indefiniteness is likely to be broader than in this benchmark. Whether the ranking of the methods carries over remains to be tested.

- **Embedding in SQP.** The QPs were solved in isolation with a Python prototype. An implementation in HPIPM and `acados` is needed to compare against SQP-level regularization in terms of SQP iterations, run time and closed-loop performance.

Bibliography

- [1] H. G. Bock and K.-H. Plitt, “A multiple shooting algorithm for direct solution of optimal control problems,” in *Proceedings of the 9th IFAC World Congress*, vol. 17, 1984, pp. 242–247. DOI: 10.1016/S1474-6670(17)61205-9.
- [2] M. Diehl and S. Gros, *Numerical Optimal Control*. –, expected to be published in 2018. [Online]. Available: <http://www.syscop.de/numericaloptimalcontrol%7D>.
- [3] C. V. Rao, S. J. Wright, and J. B. Rawlings, “Application of Interior-Point Methods to Model Predictive Control,” *Journal of Optimization Theory and Applications*, vol. 99, no. 3, pp. 723–757, Dec. 1998, ISSN: 0022-3239, 1573-2878. DOI: 10.1023/A:1021711402723. Accessed: Dec. 12, 2025.
- [4] P. T. Boggs and J. W. Tolle, “Sequential quadratic programming,” *Acta Numerica*, vol. 4, 1995. DOI: 10.1017/S0962492900002518.
- [5] R. Verschueren et al., *Acados: A modular open-source framework for fast embedded optimal control*, 2020. arXiv: 1910.13753 [math.OC]. [Online]. Available: <https://arxiv.org/abs/1910.13753>.
- [6] G. Frison and M. Diehl, *Hpipm: A high-performance quadratic programming framework for model predictive control*, 2020. arXiv: 2003.02547 [math.OC]. [Online]. Available: <https://arxiv.org/abs/2003.02547>.
- [7] A. V. Fiacco and G. P. McCormick, *Nonlinear Programming: Sequential Unconstrained Minimization Techniques* (Classics in Applied Mathematics). Philadelphia, PA: Society for Industrial and Applied Mathematics, 1990, vol. 4.
- [8] J. Nocedal, A. Wächter, and R. A. Waltz, “Adaptive Barrier Update Strategies for Nonlinear Interior Methods,” *SIAM Journal on Optimization*, 2009. DOI: 10.1137/060649513. Accessed: May 13, 2026. [Online]. Available: <https://sci-hub.st/10.1137/060649513>.

-
- [9] S. Mehrotra, “On the implementation of a primal-dual interior point method,” *SIAM Journal on Optimization*, vol. 2, no. 4, pp. 575–601, 1992. DOI: 10.1137/0802028. eprint: <https://doi.org/10.1137/0802028>. [Online]. Available: <https://doi.org/10.1137/0802028>.
- [10] D. Kiessling, S. Leyffer, and C. Vanaret, “A unified funnel restoration SQP algorithm,” *Mathematical Programming*, Oct. 22, 2025, ISSN: 1436-4646. DOI: 10.1007/s10107-025-02284-3. Accessed: May 26, 2026. [Online]. Available: <https://doi.org/10.1007/s10107-025-02284-3>.
- [11] L. Armijo, “Minimization of functions having lipschitz continuous first partial derivatives,” *Pacific Journal of Mathematics*, vol. 16, no. 1, pp. 1–3, 1966.
- [12] J. Nocedal and S. Wright, *Numerical Optimization* (Springer Series in Operations Research and Financial Engineering), 2nd. New York: Springer Science & Business Media, 2006, ISBN: 978-0-387-30303-1. DOI: 10.1007/978-0-387-40065-5.
- [13] T. McSweeney, “Modified Cholesky Decomposition and Applications,” M.S. thesis, University of Manchester, Sep. 8, 2017. Accessed: Apr. 20, 2026. [Online]. Available: <https://eprints.maths.manchester.ac.uk/2599/>.
- [14] J. J. Moré, “The levenberg-marquardt algorithm: Implementation and theory,” in *Numerical Analysis*, G. A. Watson, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1978, pp. 105–116, ISBN: 978-3-540-35972-2.
- [15] R. Verschueren, M. Zanon, R. Quirynen, and M. Diehl, “A Sparsity Preserving Convexification Procedure for Indefinite Quadratic Programs Arising in Direct Optimal Control,” *SIAM Journal on Optimization*, vol. 27, no. 3, pp. 2085–2109, Jan. 2017, ISSN: 1052-6234, 1095-7189. DOI: 10.1137/16M1081543. Accessed: Sep. 5, 2025.
- [16] P. E. Gill, W. Murray, and M. H. Wright, *Practical Optimization*. London: Academic Press, 1981.
- [17] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.

- [18] G. Frison, D. Kouzoupis, J. B. Jørgensen, and M. Diehl, “An efficient implementation of partial condensing for Nonlinear Model Predictive Control,” in *2016 IEEE 55th Conference on Decision and Control (CDC)*, Dec. 2016, pp. 4457–4462. DOI: 10.1109/CDC.2016.7798946. Accessed: Apr. 23, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/7798946>.
- [19] G. Frison, “Algorithms and Methods for High-Performance Model Predictive Control,”
- [20] H.-r. Fang and D. P. O’Leary, “Modified Cholesky algorithms: A catalog with new approaches,” *Mathematical Programming*, 2007. DOI: 10.1007/s10107-007-0177-6. Accessed: Apr. 17, 2026. [Online]. Available: <https://scihub.st/10.1007/s10107-007-0177-6>.
- [21] P.-A. Absil and A. L. Tits, “Newton-KKT interior-point methods for indefinite quadratic programming,” *Computational Optimization and Applications*, vol. 36, no. 1, pp. 5–41, Jan. 1, 2007, ISSN: 1573-2894. DOI: 10.1007/s10589-006-8717-1. Accessed: May 18, 2026. [Online]. Available: <https://doi.org/10.1007/s10589-006-8717-1>.
- [22] D. Kiessling, *Sweet-ocp: A test suite of direct optimal control problems*, version 0.1. 2026. [Online]. Available: https://github.com/david0oo/sweet_ocp.
- [23] M. W. Spong, “The swing up control problem for the Acrobot,” *IEEE Control Systems Magazine*, vol. 15, no. 1, pp. 49–55, 1995. DOI: 10.1109/37.341864.
- [24] L. E. Dubins, “On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents,” *American Journal of Mathematics*, vol. 79, no. 3, pp. 497–516, 1957. DOI: 10.2307/2372560.
- [25] K. Furuta, M. Yamakita, and S. Kobayashi, “Swing-up control of inverted pendulum using pseudo-state feedback,” *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, vol. 206, no. 4, pp. 263–269, 1992. DOI: 10.1243/PIME_PROC_1992_206_341_02.
- [26] J. T. Betts, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming* (Advances in Design and Control), 2nd. Philadelphia, PA: SIAM, 2010. DOI: 10.1137/1.9780898718577.

- [27] J. S. Meditch, “On the problem of optimal thrust programming for a lunar soft landing,” *IEEE Transactions on Automatic Control*, vol. 9, no. 4, pp. 477–484, 1964. DOI: 10.1109/TAC.1964.1105758.
- [28] R. Verschueren, M. Zanon, R. Quirynen, and M. Diehl, “Time-optimal race car driving using an online exact hessian based nonlinear MPC algorithm,” in *2016 European Control Conference (ECC)*, Jun. 2016, pp. 141–147. DOI: 10.1109/ECC.2016.7810277. Accessed: Dec. 19, 2025.

